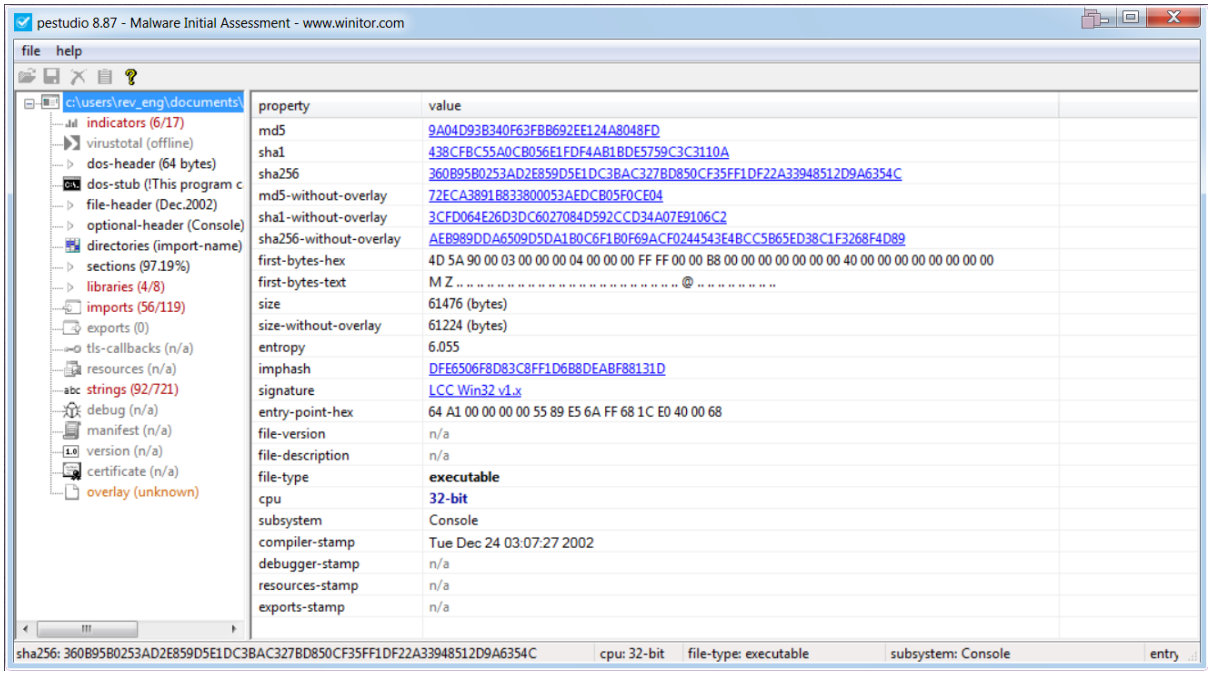
HOME EXAM

NR. 10083

1) Basic Static Analysis

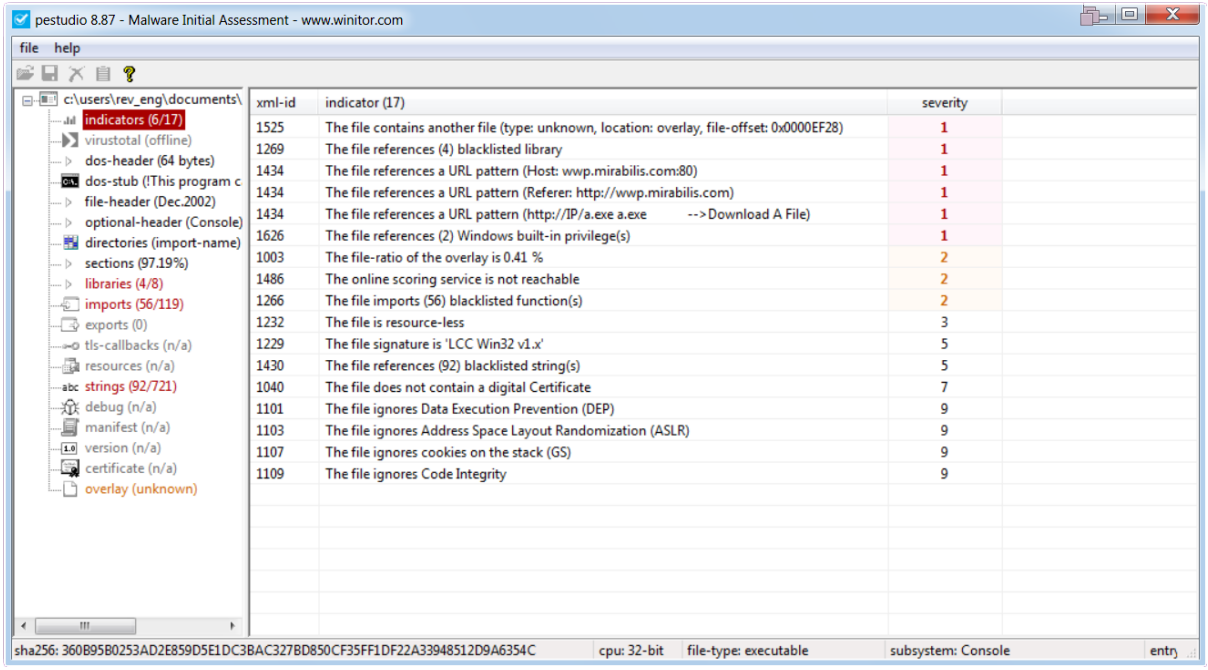
a. Malware sample: exam1a

By opening the sample in PeStudio, we find several interesting indicators (see image below). First of all, we see two properties indicating different size measurements. This is something we normally would not see for, say, a plain EXE file, where only a single size property would be displayed. Here however, one property is displaying the size without overlay, while the other is displaying the general size (including overlay). This is a clear indicator that the file contains an overlay of some sort – the purpose of which could be to obfuscate the file's real contents.



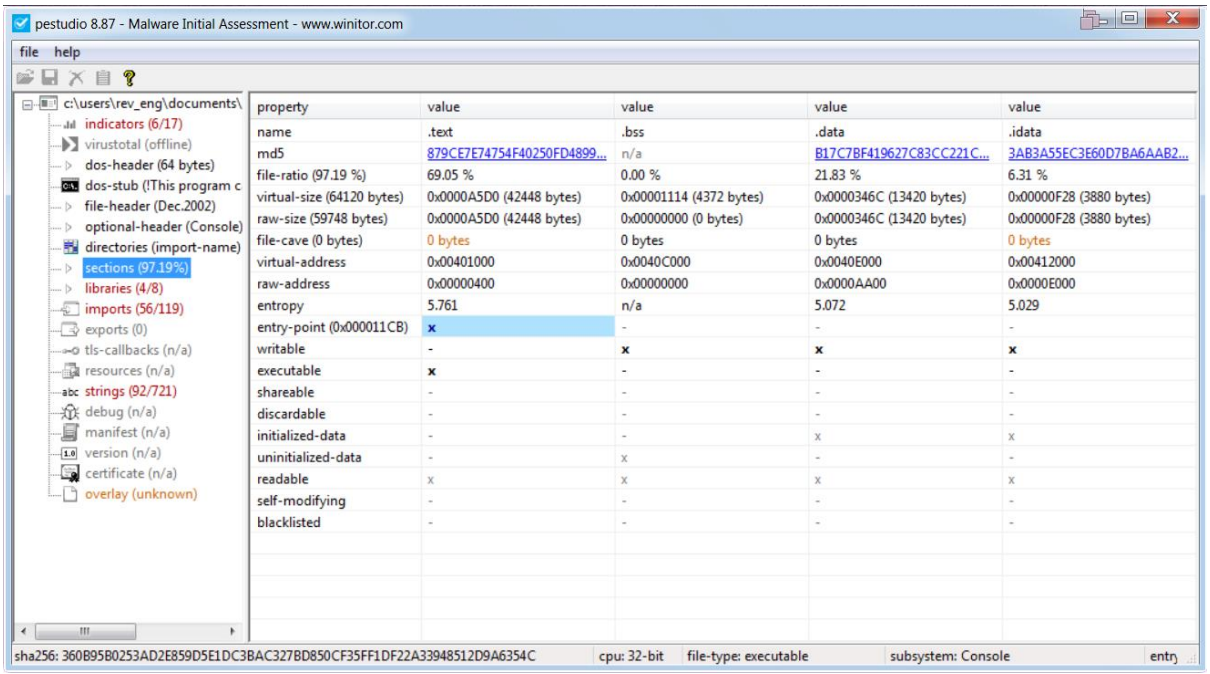
In other words, it is reasonable to assume that the actual file of interest could be hidden behind the file currently being analysed. This is supported by the fact that several pieces of important information are not available, likely due to the overlay. This includes the file version and file description, which is very general information that we would normally see. It is also worthy of note that the file was (apparently) compiled early in the morning on Christmas Eve, in 2002.

The hypothesis of a file-within-a-file is further strengthened by investigating various indicators:



Here, there are 17 indicators in total, six of which are assumed to be of malicious nature. The first indicator all but confirms that the file contains another file, starting at the offset 0xEF28. (I will not go through the rest of these indicators, as they generally give away information without the need to explain or digging deeper into the sample's structure and functionality.)

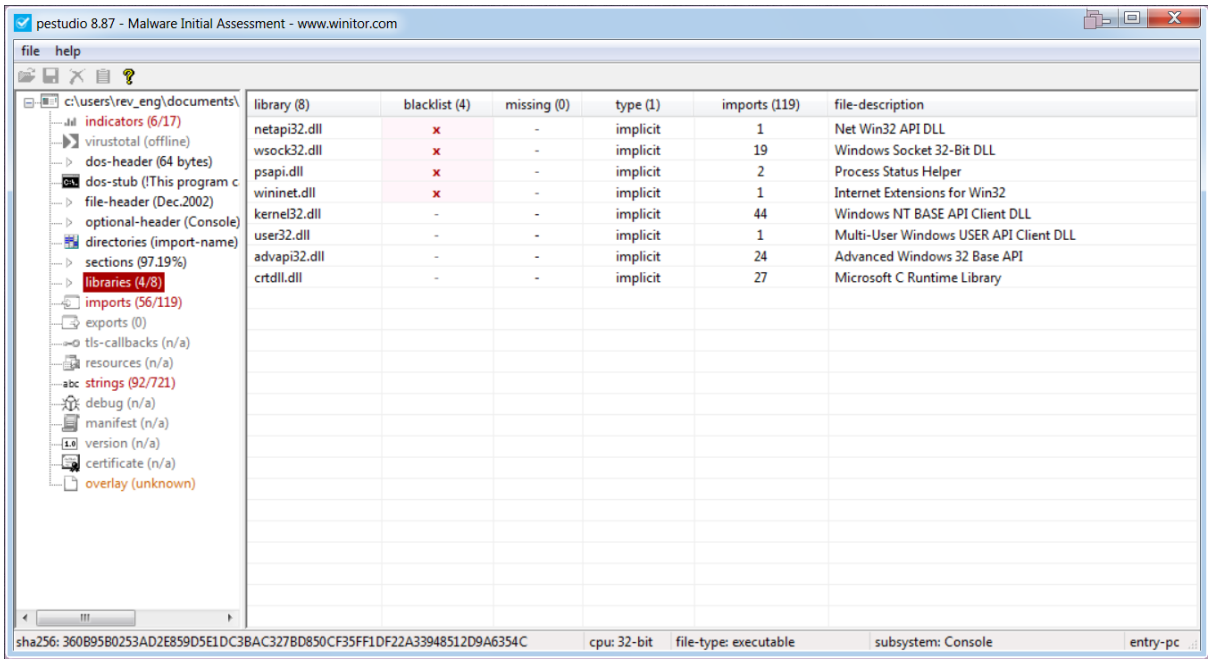
Upon inspecting the sample's file sections (see image below), it becomes evident that the virtual size is identical to the raw size for all but one section – the .bss section. This does not necessarily indicate something suspicious, as this particular section normally contains statically allocated variables that have not been assigned values yet. Because this would happen in memory, the explanation behind why the virtual size (memory) is so much bigger than the raw size (disk) for this section might not be too mysterious or of particular importance after all.



However, the identical sizes of the other sections indicates that the sample takes up the same size in memory as on disk, which is a clear indicator that the file is not packed. It is important here to distinguish between 'packed' and 'obfuscated'; obfuscation is a broader term, meaning packing is a subset of obfuscation. Therefore, the file could be obfuscated without being packed. This appears to be the situation here, since we are able to analyse large parts of the file, which we would not be able to if the file was packed and compressed. It is however reasonable to assume that the file is obfuscated in some other way, as already addressed with the overlay and idea of a file-within-a-file. (By right clicking the sample and choosing 'Properties', we also see that the file's size (60KB) differs from the size on disk (64KB).)

Imports

The file includes a great deal of imported functions, through the libraries listed below:



Libraries such as NETAPI32.DLL, WSOCK32.DLL and WININET.DLL all indicate that the sample performs network-related activity. This theory is supported by investigating imports (under the "imports" section in PeStudio, shown directly below "libraries" in the image above), where functions such as "InternetGetConnectedState", "socket", "send" and "listen" are found.

Furthermore, other imported functions make it evident that the sample is able to create, open and manipulate crucial elements of the system – including, but not limited to:

- Registry: RegCreateKey, RegOpenKey, RegSetValue
- Services: CreateService, StartService, DeleteService
- Threads: CreateThread, ExitThread, TermnateThread
- Processes: CreateProcess, OpenProcess, TerminateProcess
- Files: ReadFile, WriteFile, DeleteFile, FindFirstFile, FindNextFile

Strings

The file contains a tremendous number of interesting strings, where I will attempt to address some of the most notable ones.

This includes email references, such as:

- MAIL FROM:Victim@hotmail.com
- mc3.law16.hotmail.com
- mc2.law16.hotmail.com
- mc1.law16.hotmail.com

This could indicate that the sample notifies the attacker of infection on the victim's system, where the last three lines could resemble an email address used by the attacker.

Several strings could further indicate that the sample attempts to download a file and execute it on the infected system:

- http://IP/a.exe a.exe -->Download A File
- Execute Program -->Execute A Program

In addition, some strings could indicate that the sample attempts to change the victim's passwords, likely in order to gain unauthorized access to other accounts or parts of the infected system:

- The User(%s) Password Has Been Changed
- The Password Has Been Set To %s Successfully
- Enter Password:

Numerous strings also support the theory that the sample manipulates services, processes and threads, as well as performs network activity with port handling, downloads and installations.

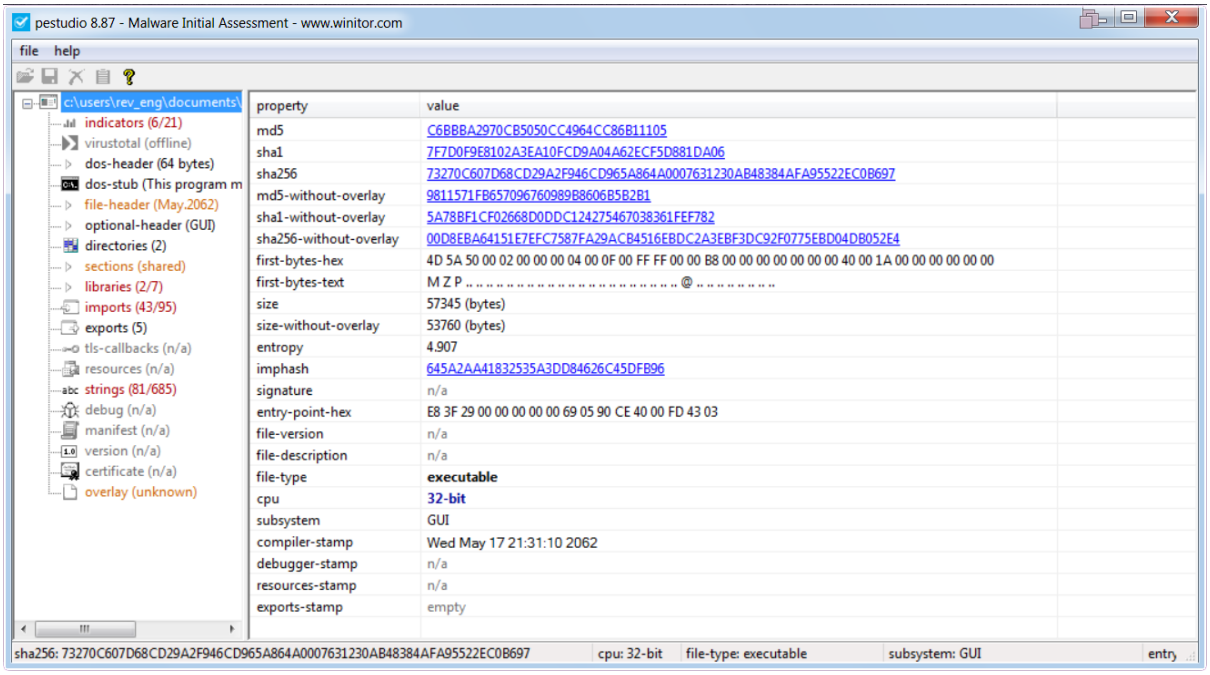
Conclusion

Based on these findings, it is reasonable to conclude that the sample possesses extensive capabilities related to networking, registry, services, processes, threads, and files. It includes several references to emails, usernames, and passwords, as well as starting and stopping of services. On the whole, it seems likely that dynamic analysis would give a clearer picture of what is going on, by examining process manipulation and network activity. For now, though, a logical hypothesis is that the sample uses the capabilities described to gain access to and alter user-sensitive data, while also attempting to obfuscate itself through the use of an overlay.

b. Malware sample: exam1b

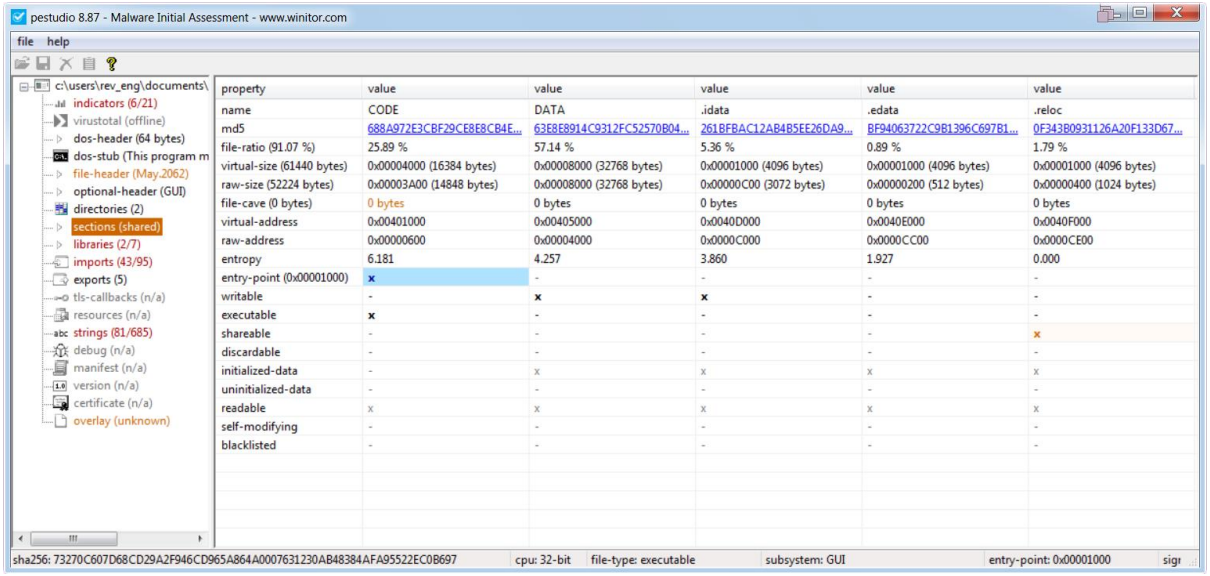
i. Find and explain abnormalities based on basic static techniques, without including imports and string.

Note: As with the previous task, I am assuming here that the "indicators" tab should not really be used, as it gives away most of the key functionality pertaining to the sample.



The first thing we notice here is that, as with the first sample, there is one general size and one size without overlay, where the former is larger than the latter, as it most likely includes the overlay. As before too, there is no available file version or file description – or even signature, as shown above. An interesting giveaway on this occasion though, is the compiler stamp. Here, it is set to May 17th, 2062 – a year far into the future, clearly signifying that the compilation date has been manipulated by the author of the sample. This should be considered an abnormality, and even malicious indicator, as a benevolent file author would have little reason to manipulate the date of compilation, let alone to set it almost 40 years from now.

By examining the sizes of the different file sections, we see another abnormality. Every section, except for the DATA section, differs in virtual and raw size. The virtual size is always higher (equal for DATA), which tells us that the sample takes up more size in memory than on disk.



ii. Based on strings only; which strings are unique to this sample and could be used, alone or together, in a signature to detect this particular sample.

Note: I assume here that this could be any strings connected to the sample (but, as clearly stated in the task, strings only), and not those found under the "strings" section.

Multiple strings are unique to this sample, mostly those representing hash values, including:

- MD5 string: C6BBBA2970CB5050CC4964CC86B11105
- SHA-1 string: 7F7D0F9E8102A3EA10FCD9A04A62ECF5D881DA06
- SHA-256 string: 73270C607D68CD29A2F946CD965A864A0007631230AB48384AFA95522EC0B697

The strings representing corresponding hash values for the file without overlay could likely also be used in a unique signature to detect this particular sample – but not alone, since they would only be hashes of *part* of the sample, that is, the part of the sample that excludes the overlay.

iii. Write a hypothesis of the potential capability and purpose of the sample based on minimum 5, but maximum 10 indicators.

Note: I assume that indicators can be found by examining the "strings" and "imports" sections.

Indicators related to the sample are as follows (strings and imports represent one each, both with corresponding examples to illustrate why I believe they constitute indicators in this case):

- Contains an overlay.
- Suspect compilation date.
- Varied size of raw and virtual data.
- Exports related to virtualization and threads.
- Imported functions, related to:
  - Anti-debugging: GetTickCount
  - Files: CreateFile, ReadFile, WriteFile, etc.

    - Registry: RegCreateKey, RegOpenKey, RegSetValue, etc.
    - Network: HttpSendRequest, InternetConnect, InternetOpen, etc.
    - Processes: CreateProcess, OpenProcess, GetCurrentProcessId, etc.
    - Keys: GetKeyboardState, GetForegroundWindow, GetWindowText, etc.
- Questionable strings, such as:
    - ===FORMS LOG START===
    - ===KEYLOGGER DATA START===
    - ===CLIPBOARD LOG===
    - mailsended

Based on these indicators, a hypothesis is that the sample represents a malicious keylogger. It appears to store the victim's keyboard input to a file and possibly sends the recorded data through networking means, such as e-mail. This is strongly supported by the combination of a dubious compilation date, varied sizes of sections on disk and in memory, suspicious strings including words like "keylogger", and the file-, network- and key-related functions listed above.

It is also fairly likely that the sample observes whether it is being analysed or detected, by using anti-debugging functions and exports to check whether the infected system is a virtualization environment. This theory is further strengthened by the many appearances of strings related to antivirus services ("127.0.0.1 kaspersky.com", etc.), which could indicate that the sample tries to determine the current antivirus/antimalware solution installed on the infected system.

Additionally, it is possible that the sample tries to circumvent or prevent the detection of its presence by obscuring itself through the overlay functionality/mechanism already described.

2) Basic Dynamic Analysis

a. Malware sample: exam2a

i. Using Regshot: What files and folders are added by this malware?

The following files were added by this malware:

- C:\Users\rev_eng\AppData\Local\Temp\edakubnfgu
- C:\Users\rev_eng\AppData\Local\Temp\gwhroqkhwu
- C:\Users\rev_eng\AppData\Local\Temp\klibvpszzn
- C:\Users\rev_eng\AppData\Local\Temp\phqghumeay
- C:\Users\rev_eng\AppData\Local\Temp\rgjqmvnkyr
- C:\Windows\M-5050396903767369379305959 40\winmgr.exe

The following folder was added by this malware:

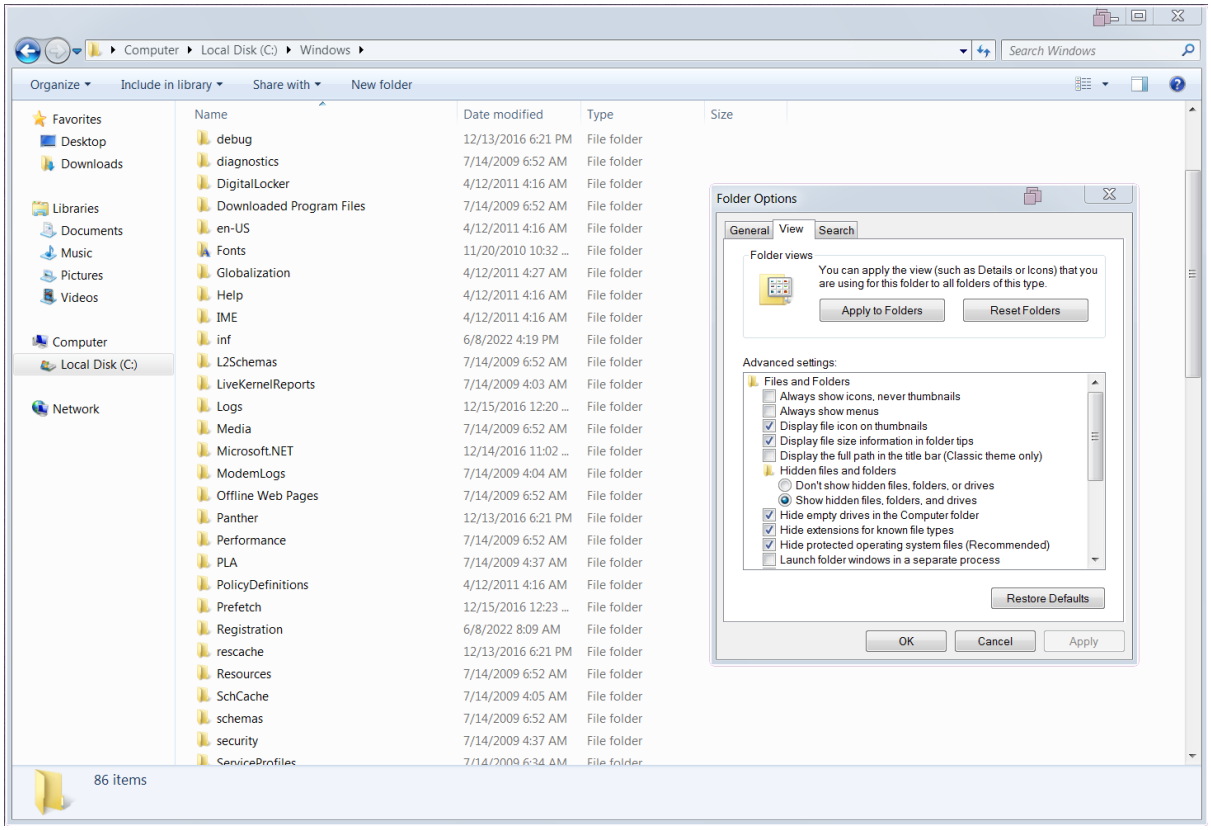
- C:\Windows\M-505039690376736937930595940

ii. Using Regshot: What files and folders are deleted by this malware?

The following file was deleted by this malware (no folder was deleted):

- C:\Users\rev_eng\Documents\Exam Samples\exam2a.exe

Interestingly, because the folder containing the file 'winmgr.exe' was created, and not deleted, it would normally be visible when navigating file explorer. However, even after making sure hidden files and folders are shown, it is not displayed in the folder where it seemingly was created, seen below. This could indicate something to keep an eye on as the analysis continues.



iii. Using regshot: Explain any registry values added, that an analyst should be aware of? Specify what value and why you think it is of interest.

The malware adds two similar registry values that are of particular interest:

- HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Run\Microsoft Windows Manager: "C:\Windows\M-505039690376736937930595940\winmgr.exe"
- HKU\S-1-5-21-3627035991-92649225-2619829572-1000\Software\Microsoft\Windows\CurrentVersion\Run\Microsoft Windows Manager: "C:\Windows\M-505039690376736937930595940\winmgr.exe"

The former is added to the Local Machine (HKLM) registry hive, while the latter is added to the Users (HKU) registry hive. They are near-identical though, as the purpose of both additions is likely to increase the malware's persistence. More specifically, these values are of particular interest since they indicate that the malware has made sure it will run at system start-up. This can be understood since the path of the file in the registry value corresponds with the path of the seemingly most interesting file (listed last in task a.i.) added by the malware – 'winmgr.exe'.

iv. *Open the files from the first questions (i) in PeStudio. What can you say about similarities or differences?*

Note: I assume here that the files in question are the five similar non-EXE files, listed in task a.i., since the EXE file is not visible in file explorer or when attempting to open it in PeStudio.

All five non-EXE files appear to include the exact same sections, as well as very similar imports and strings, with near-identical amounts. Yet another similarity is that each file appears to contain another file through an overlay. Interestingly, the sizes without overlay are identical, but the general sizes differ slightly. This clearly indicates that the overlay itself is what separates the files from each other, perhaps in order to generate different hashes and seemingly entirely different files – when in reality, behind each individual/different overlay, is the same file.

The connection between these similar files and the EXE file is also evident, since strings such as the following can be found when inspecting the "strings" section of the files in PeStudio:

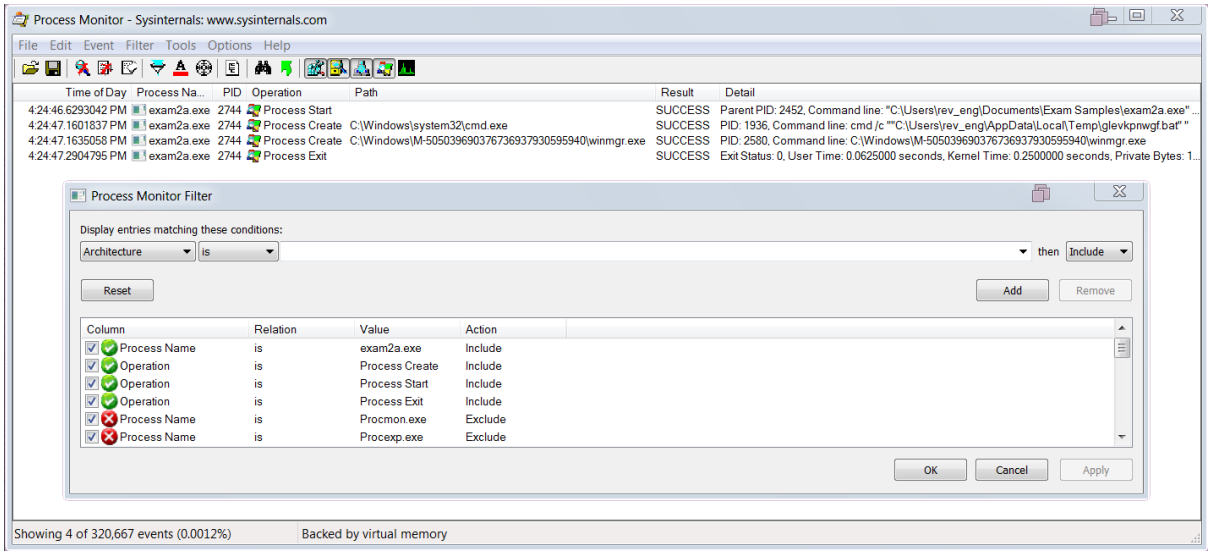
- winmgr.exe
- %s\winmgr.txt
- If Process.Name = "winmgr.exe" Then

There are other noteworthy entries among the strings, such as IP addresses and obscure DLLs:

- 220.181.87.80
- WPESPY.DLL
- DIR_WATCH.DLL

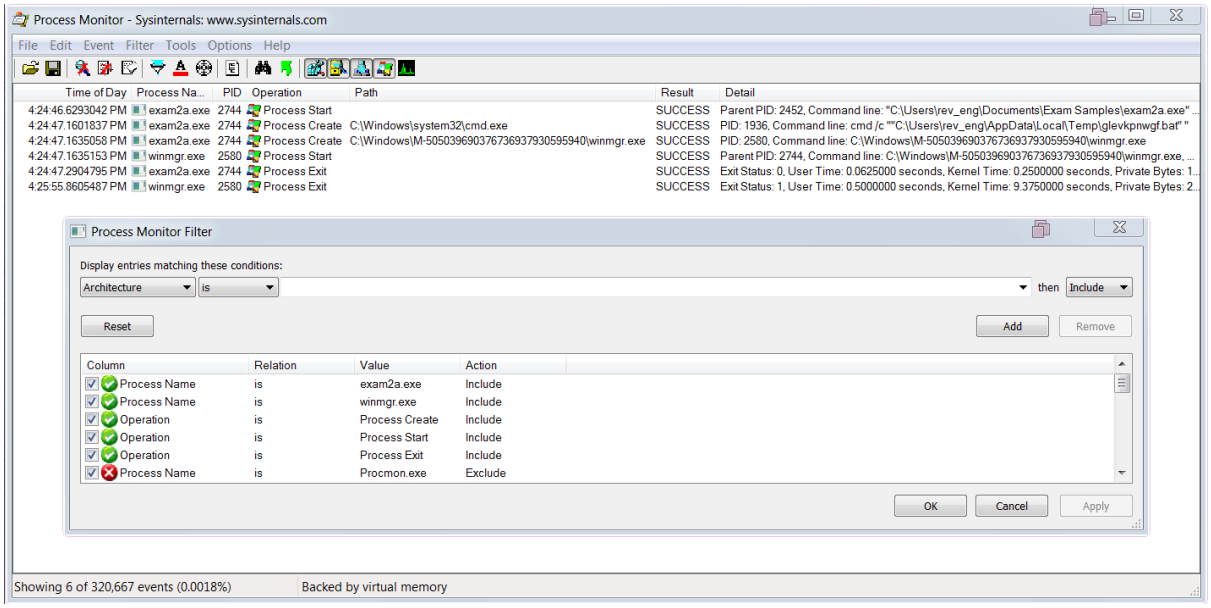
v. *Using process Monitor: Describe what processes are started, created and stopped as a result of executing the malware? Point out any new info revealed.*

By applying the filter shown below, the following processes are started, created and stopped:



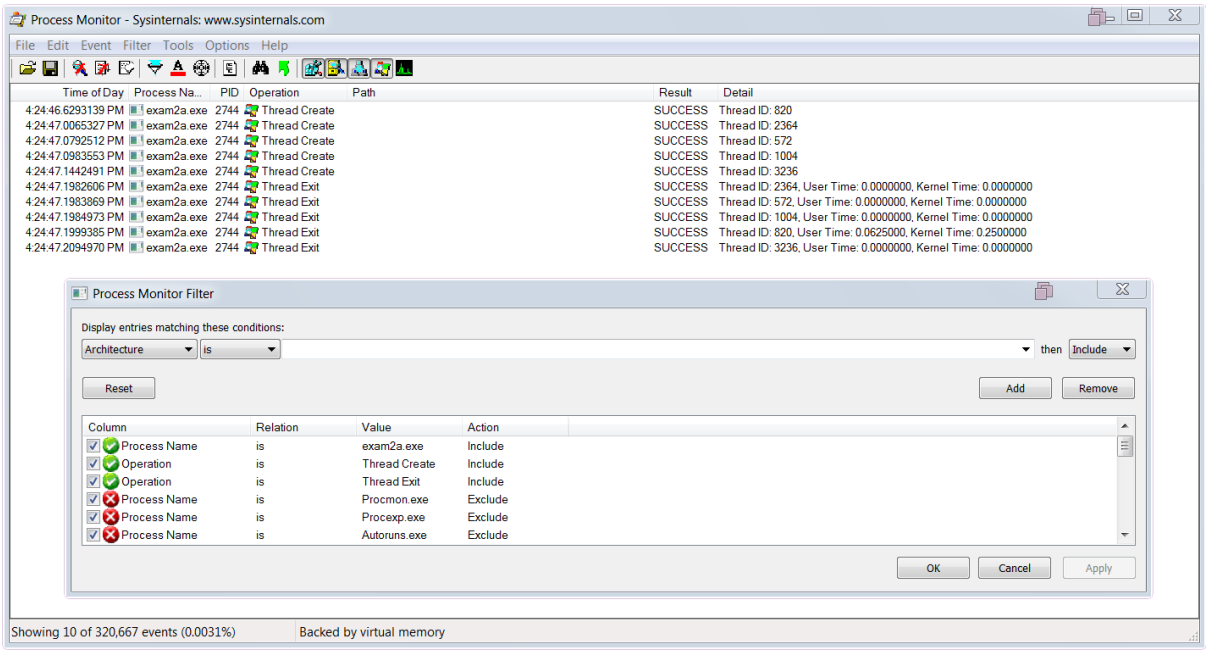
The sample executed shows up as the process being started, but more interesting are the next two events. Here, through the command prompt, a BAT file is run, which is most likely a script containing a series of commands to be executed. This in turn appears to launch the EXE file we have seen earlier – 'winmgr.exe'. Finally, the originally executed sample exits itself, leaving the newly created process running. These events likely take place to allow the malware to run under a different name than the originally executed sample, which was named 'exam2a.exe', as opposed to 'winmgr.exe', and to potentially escalate its own privilege, since it has now successfully executed from a sub-directory located in 'C:\Windows\', instead of 'C:\Users\'.

By now including the newly created process in the filter, we can see that it starts shortly after creation, and finally is exited, which notably is a result of manually terminating the malware with Process Hacker, as seen by the fact that the event takes place about a minute after the previous events (the recommended runtime was 30 seconds, but I let it run a little longer).

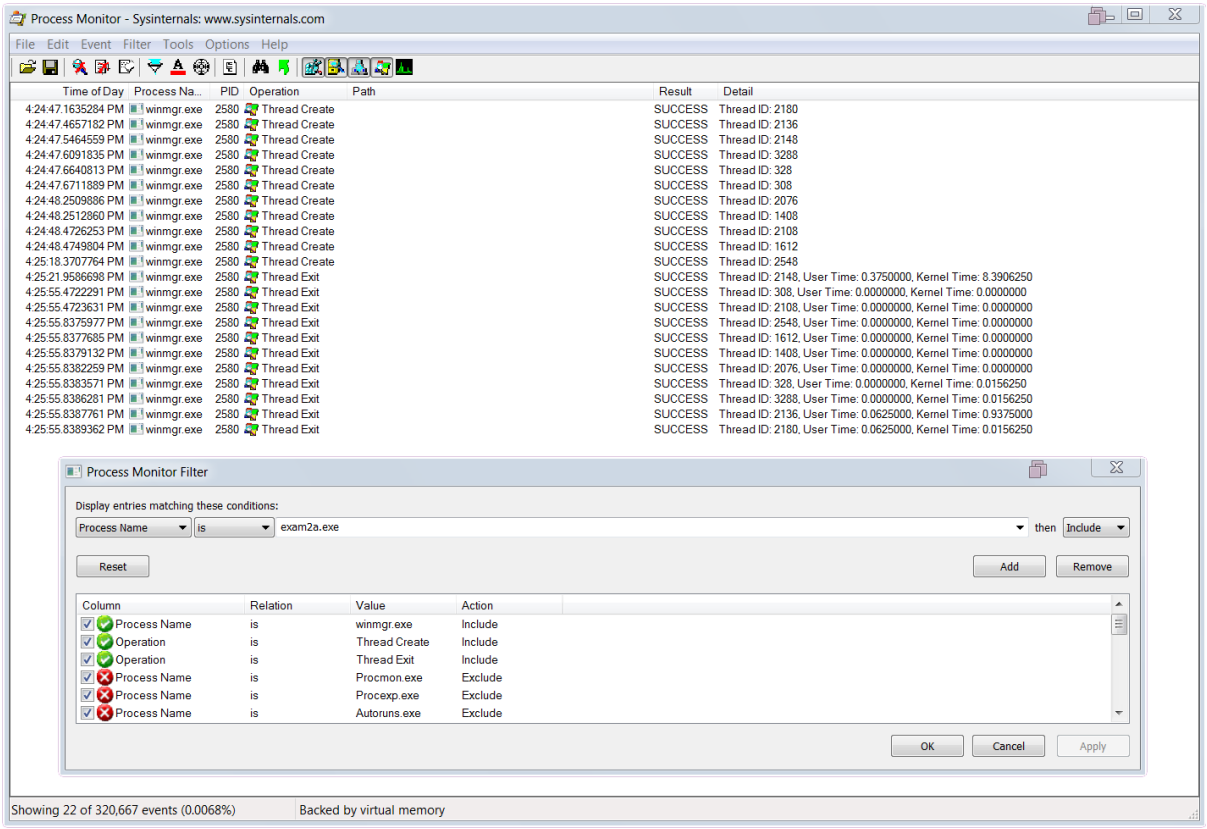


vi. *For each of the processes found above (v), how many threads are created and stopped. How many would still be running if you did not terminate the malware and what process(es) do they belong to?*

For the first process, 'exam2a.exe', five threads are created and stopped, as seen below:



For the second process, 'winmgr.exe', eleven threads are created and stopped, as seen below:



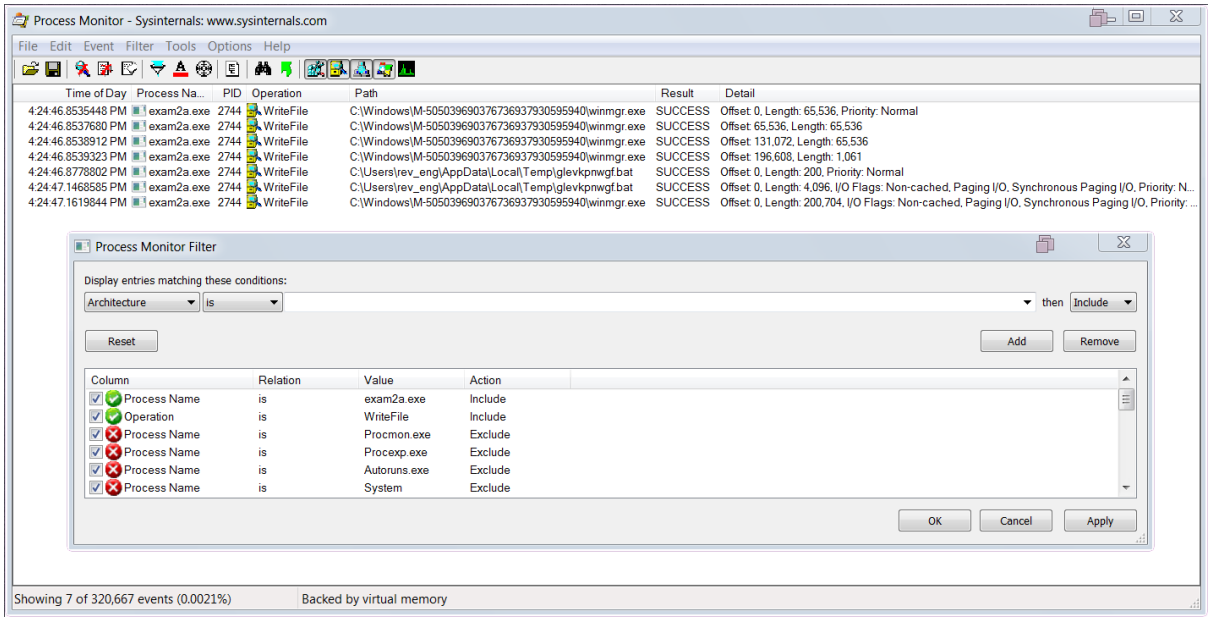
If I did not terminate the malware, none of the threads belonging to the first process would be running even then, as that process terminates itself, as verified by the time of those thread-exit events (around 4:24:47), set well before the termination of the malware (around 4:25:55).

As for the threads belonging to the second process, there is reason to believe that all but one of the threads would still run if I did not terminate the malware. The one thread that would certainly exit regardless has ID 2148, and clearly exits well before the manual termination. The other ten threads are very close to the termination time, and it is therefore logical to assume that they could potentially continue to run if termination of the malware did not take place.

In summary, the ten threads that would likely continue to run all belong to the second process, which itself would naturally also continue to run if not for the termination of the malware.

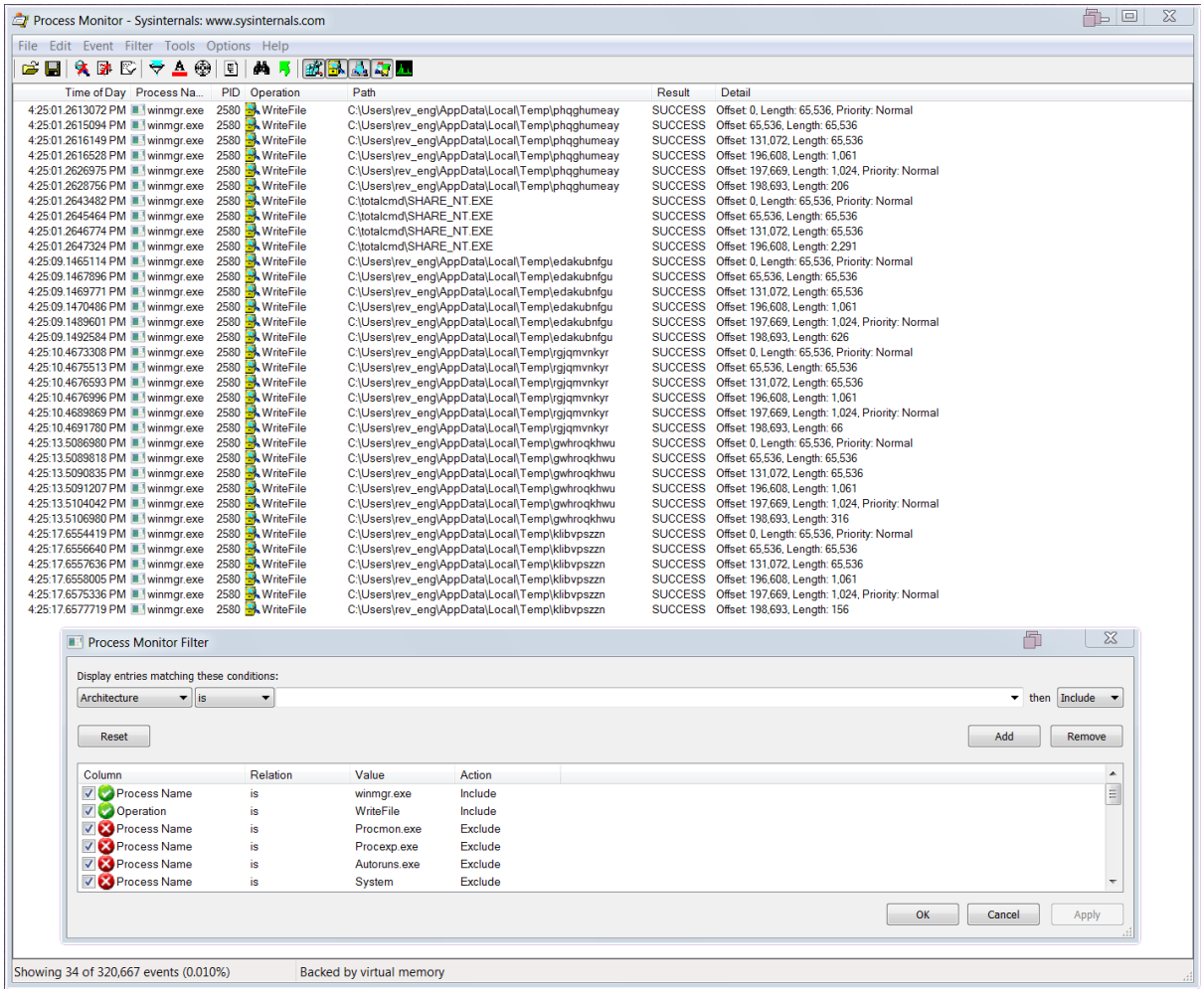
vii. For each of the processes found above (v), what files are written to. Point out any new information revealed.

The first process writes to two different files, as seen below:



While there are seven entries, only two different files are written to. They are therefore written to on numerous occasions. The first file is what the (previously discussed) self-created process executed – 'winmgr.exe'. The last file is the also mentioned BAT file that likely serves as a script.

The second process writes to six different files, as seen below:



As with the previous process, all six files are written to on numerous occasions, hence more than six events. Five of these files are familiar, which were displayed by Regshot and later discovered to be largely similar files. However, the last one appears to reveal new information, indicating that a file located at 'C:\totalcmd\SHARE_NT.EXE' has also been written to.

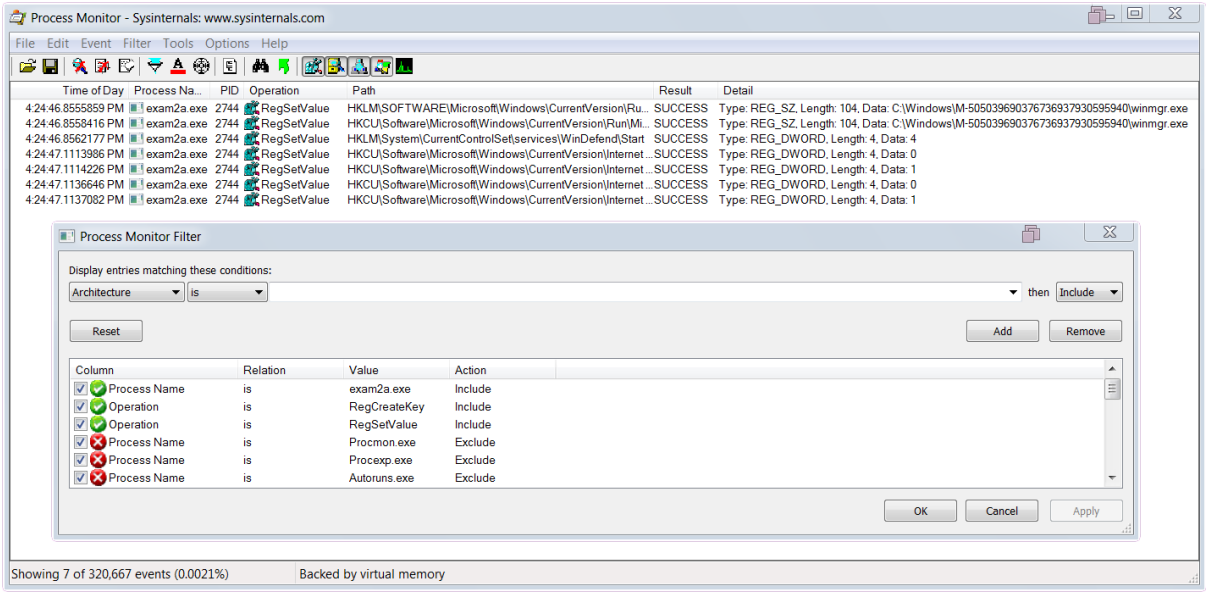
The exact purpose of this file is not clear, but there is reason to believe that the malware has taken advantage of the install location of the program 'Total Commander'. This program in itself is not malicious, but perhaps serves as something the malware attempts to exploit for the sake of possible motivations such as increased persistence or escalated privileges.

Upon further inspection, it becomes clear that 'SHARE_NT.EXE' indeed has a near-identical file size to the other five files*. This means it likely has the same malicious properties, and despite the EXE file format, there is reason to believe that this is a near-copy of the other files, perhaps the main difference being that this file does not include the overlay which the other five files did. This also amplifies the theory that the other five files are indeed EXE files "under the hood", and that they in reality simply are obfuscated/masked by the mentioned overlay.

* Identical size on disk.

viii. For each of the processes look at ***"registry keys created" and "registry values set"****. For all entries that are not network related, explain what they achieve.*

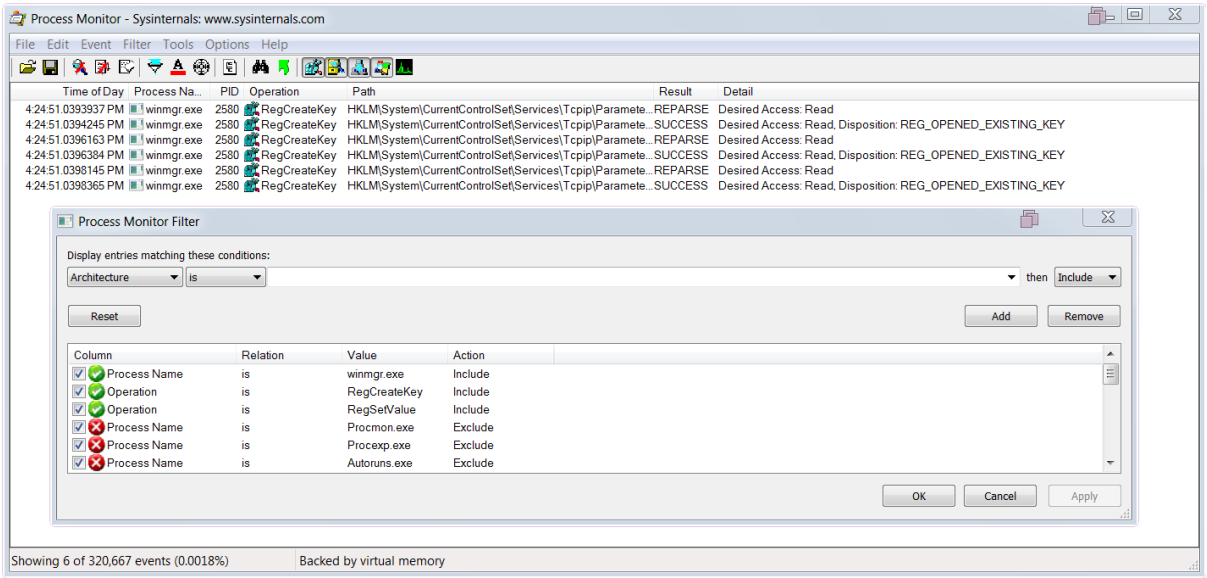
The first process performs the following registry activity, all related to setting values:



The first two entries ensure the malware is run at system start-up. As seen under the "Detail" column, the file created by the malware itself is run every time, and not the originally run sample (which was deleted after malware execution, as addressed in the Regshot part earlier).

The third entry appears to disable Windows Defender, by setting its 'Start' value to 4. (This answer helped in figuring out the meaning behind the value 4.) This of course poses a major concern, where the malware is clearly attempting to impair the security of the victim system.

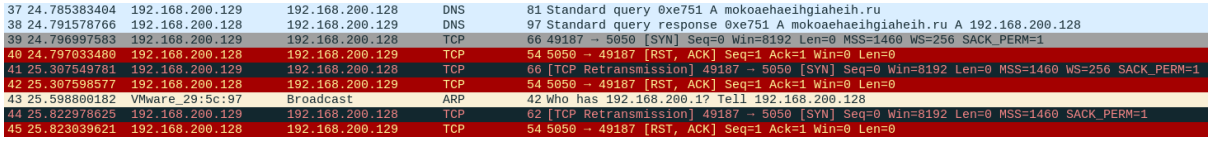
The second process performs the following registry activity, all related to creating keys:



These all appear to be network-related (TCP/IP), and I will therefore not explain their purpose.

ix. Describe any network traffic you detect, include relevant screenshots.

Investigation of network traffic in Wireshark shows that the following pattern repeats itself:



37	24.785383404	192.168.200.129	192.168.200.128	DNS	81 Standard query 0xe751 A mokoaehaeihgiaheih.ru
38	24.791578766	192.168.200.128	192.168.200.129	DNS	97 Standard query response 0xe751 A mokoaehaeihgiaheih.ru A 192.168.200.128
39	24.796997583	192.168.200.129	192.168.200.128	TCP	66 49187 → 5050 [SYN] Seq=0 Win=8192 Len=0 MSS=1460 WS=256 SACK_PERM=1
40	24.797033480	192.168.200.128	192.168.200.129	TCP	54 5050 → 49187 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
41	25.307549781	192.168.200.129	192.168.200.128	TCP	66 [TCP Retransmission] 49187 → 5050 [SYN] Seq=0 Win=8192 Len=0 MSS=1460 WS=256 SACK_PERM=1
42	25.307598577	192.168.200.128	192.168.200.129	TCP	54 5050 → 49187 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
43	25.598800182	VMware_29:5c:97	Broadcast	ARP	42 Who has 192.168.200.1? Tell 192.168.200.128
44	25.822978625	192.168.200.129	192.168.200.128	TCP	62 [TCP Retransmission] 49187 → 5050 [SYN] Seq=0 Win=8192 Len=0 MSS=1460 SACK_PERM=1
45	25.823039621	192.168.200.128	192.168.200.129	TCP	54 5050 → 49187 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0

The pattern generally starts off with a DNS request sent from the infected Windows machine (IP 192.168.200.129) to the REMnux machine (IP 192.168.200.128). In this case, the requested domain name is the fairly suspicious “mokoaehaeihgiaheih.ru”, which seems to constitute an unknown Russian website. (In response, the REMnux machine simply replies that the domain is associated with its own local IP address, which in reality is obviously not the case, though presumably done here by INetSim for simulation purposes.) Note that the exact domain name varies per request, but they are consistently suspect and Russian. These most likely manifest malicious websites the infected system tries to reach once compromised by the malware.

Ignoring the ARP request, the remaining lines represent an attempted TCP handshake between the Windows machine and the REMnux machine. The pattern indicates that the handshake between the two is not succeeding. The port used is consistently 5050, but it appears that whatever service is listening on this port is not properly simulated. Upon further inspection, the service used for this port appears to be ‘Yahoo! Messenger’, which could indicate that the malware is using this specific service to send/receive messages to/from the infected system.

b. Malware sample: exam2b

i. What file(s) and folders are added by this malware?

Note: I have used Regshot here as well.

The following files were added by this malware:

- C:\Users\rev_eng\AppData\Roaming\Microsoft\Windows\WBR9SSKCN.dat
- C:\Windows\InstallDir\ActiveX.exe

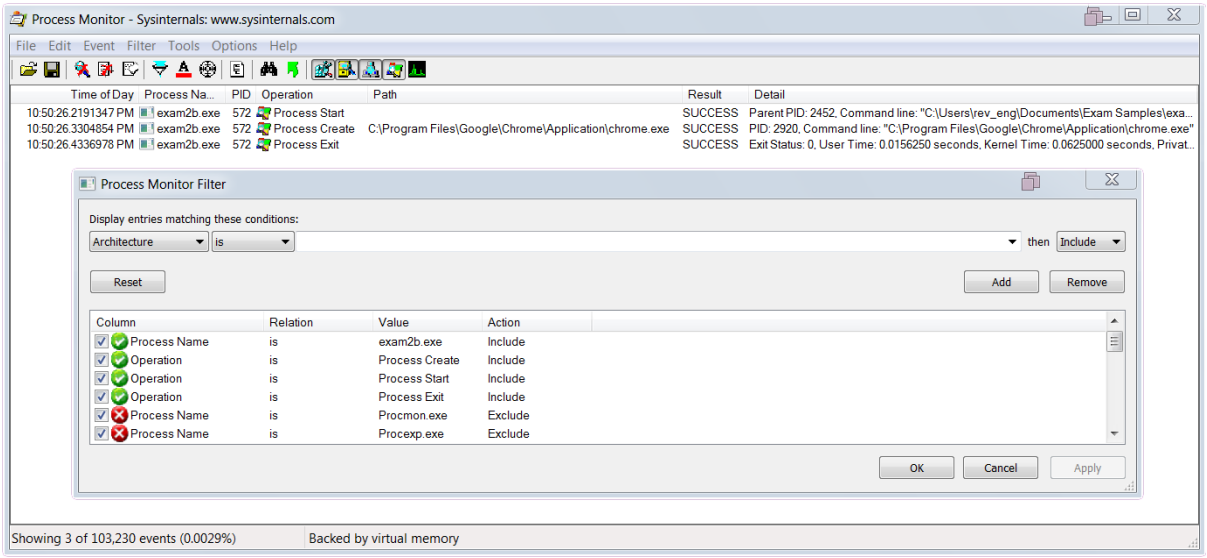
The following folder was added by this malware:

- C:\Windows\InstallDir

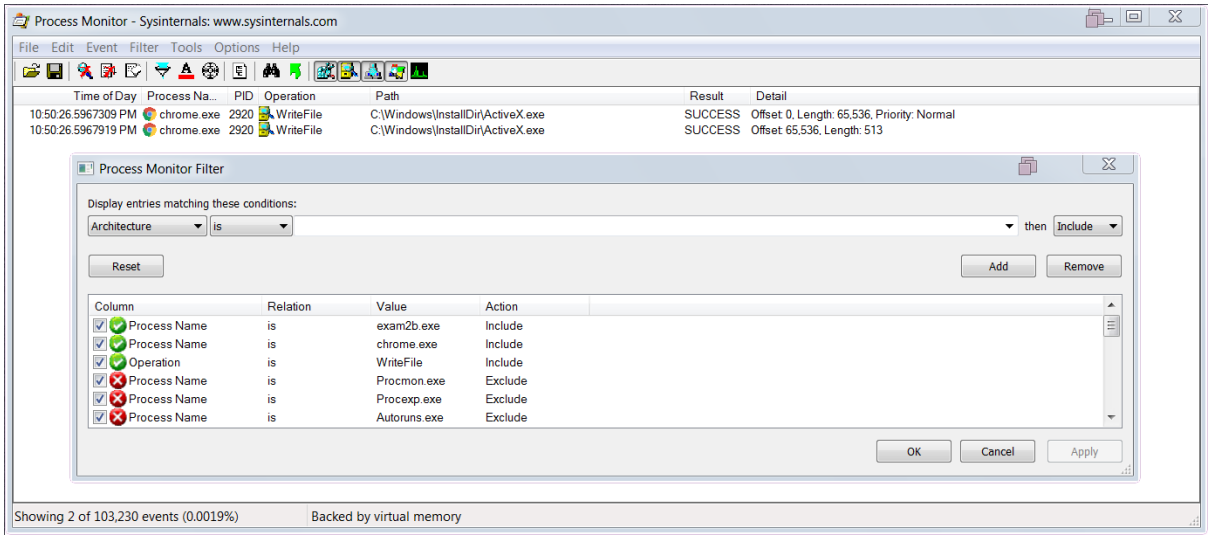
ii. What process creates the file(s)?

By inspecting files written to in Process Monitor, it becomes evident that ‘chrome.exe’, that is, Google Chrome, has created the file ‘ActiveX.exe’. This corresponds to how, immediately after executing the malware, Process Hacker displays the new process ‘chrome.exe’. In other words,

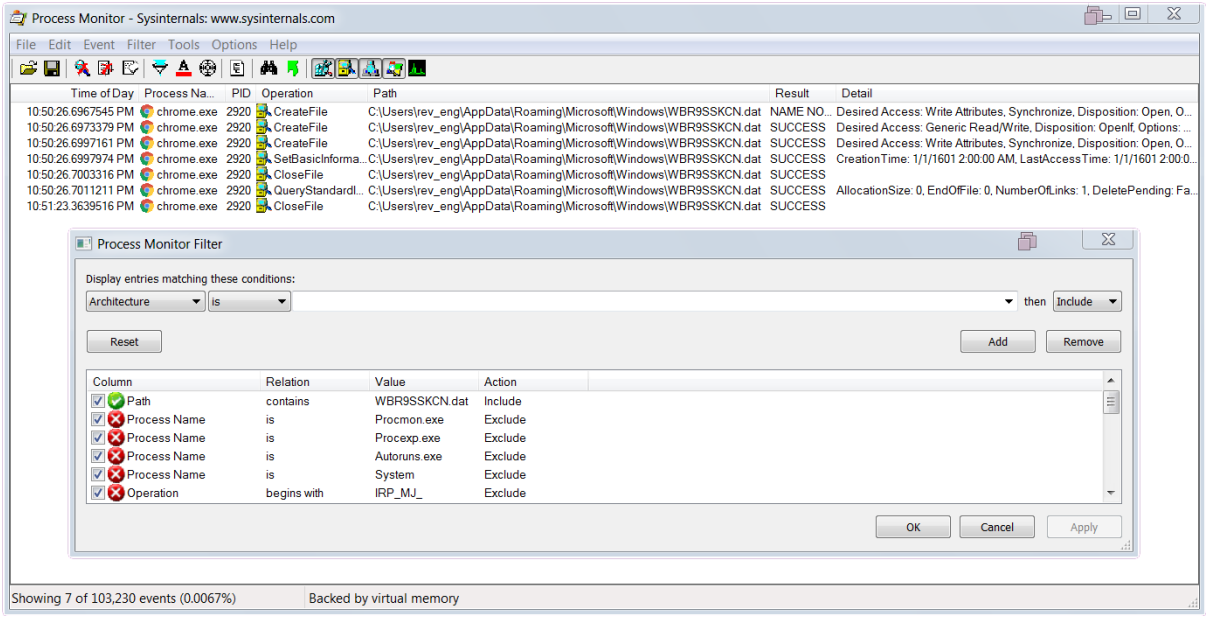
it seems as if this sample launches itself under a Google Chrome cover, pretending to be the well-known browser, and later exiting the original process, in a similar manner to the previous sample. This is verified by filtering on the sample name and various process-related operations:



We can then filter on only the process name 'chrome.exe' and operation 'WriteFile' to remove unnecessary noise from the Process Monitor window (without the process name filter, and only a 'WriteFile' filter, 30 events are shown, but we are of course not interested in all of them):



Figuring out what process created the DAT file is less straightforward than simply filtering on the files that have been written to and investigating the digestible number of events that appear. This is because the file is seemingly never written to, and we must therefore use another method to figure out what process created it. Since we already know the path of the file, we can utilize this by filtering on path, as shown below:



Here, the path is set to contain the DAT file, and the resulting entries are few and readable. The idea that the file was never written to seems to be correct, and this file also appears to have been created by the new Chrome process.

iii. What is the purpose of the file(s)?

The registry activity from the Regshot comparison displays data that could explain (part of) the purpose of both files. The following **keys** are among those added after running the malware:

- HKU\S-1-5-21-3627035991-92649225-2619829572-1000\Software\WBR9SSKCN
- HKU\S-1-5-21-3627035991-92649225-2619829572-1000\Software\XtremeRAT

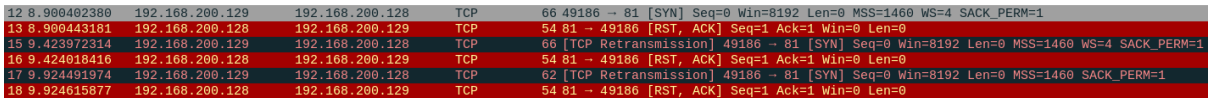
Furthermore, the following **values** are among those added after running the malware:

- HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Run\HKLM: "C:\Windows\InstallDir\ActiveX.exe"
- HKU\S-1-5-21-3627035991-92649225-2619829572-1000\Software\Microsoft\Windows\CurrentVersion\Run\HKCU: "%ActiveX%"
- HKU\S-1-5-21-3627035991-92649225-2619829572-1000\Software\WBR9SSKCN\ServerStarted: "08/06/2022 22:50:26"
- HKU\S-1-5-21-3627035991-92649225-2619829572-1000\Software\XtremeRAT\Mutex: "WBR9SSKCN"

From the latter list (values added), we see that the first two are related to the 'ActiveX.exe' file, while the last two are related to the 'WBR9SSKCN.dat' file. The former values seem to revolve around making the malware more persistent, by ensuring it always runs at start-up. The latter values seem to revolve around making sure the malware runs only if it does *not* do so already, by utilizing a mutex. Ergo, it appears that the first file, 'ActiveX.exe', is the actual malware, set to be run every time the infected system starts up, while the second file, 'WBR9SSKCN.dat', is used to check/verify when the malware was last started, effectively acting as a mutex.

iv. Describe any network traffic you detect.

Investigation of network traffic in Wireshark shows that the following pattern repeats itself:



The lines above represent an attempted TCP handshake between the Windows machine and the REMnux machine. The pattern indicates that the handshake between the two is not succeeding. The port used is consistently 81, which appears to be used in the context of onion routing, specifically for a web browser designed to run on the Tor anonymity network. The fact that the malware is trying to communicate over this type of network is a clear indication of malevolence, assuming that a benevolent piece of software would normally not attempt this.

Right before the very first handshake attempt, there is also a DNS request to the domain name 'by2olak.no-ip.biz'. Since the port used immediately after (81) is commonly related to onion routing, it is reasonable to assume that this link is some variation of an onion link, perhaps an alias or redirecting link (assuming most onion links end with '.onion'). Unlike with the previous sample though, it only happens once in this case, as opposed to before every handshake.

v. What is the mutex for the malware?

As discussed, the mutex for the malware appears to be related to the name of the created DAT file, specifically 'WBR9SSKCN.dat'. After running the malware, we saw in task iii. that several registry values were added. One of these new entries clearly implies what the mutex could be:

> HKU\S-1-5-21-3627035991-92649225-2619829572-1000\Software\XtremeRAT\Mutex: "WBR9SSKCN"

Indeed, it looks as if the mutex is "WBR9SSKCN", but precisely what this signifies is less certain. Investigation through advanced analysis would help clarify this further. In any case, the mutex is most likely used to prevent multiple instances of the malware from running at the same time.

3) Advanced Static Analysis

a. Malware sample: exam3

i. Go to address 0x1314A850: Explain each of the three first lines, and what are these together called?

```
0x1314A850                    push    ebp
0x1314A851                    mov     ebp, esp
0x1314A853                    sub     esp, 18h
```

The first line saves the base pointer (EBP) by pushing it onto the stack. The second line copies the stack pointer (ESP) to the base pointer. The third line prepares/grows the stack by allocating/reserving space for local variables to be used within the called function. In this case, 18h is used, which is the hexadecimal value 18, equal to the decimal value 24. Because each variable takes up 4 bytes, this means space is reserved for 24 / 4 = 6 variables.

Together, these three lines are commonly known as a function prologue. The purpose of the prologue is generally to prepare the stack and registers for transfer of control to a function.

ii. Go to 0x1314A858: What happens when the CALL instruction is executed (ignore the specific windows API)?

When the CALL instruction is executed, the program enters the called function – in this case GetModuleHandle – and transfers program control to this function. After the called function's instructions have completed, the program exits this function, and returns to the caller function.

iii. Go to 0x1314A85E: What happens when this instruction is executed?

```
0x1314A85E                    mov     [ebp+lpAddress], eax
```

Moves the contents of the EAX register into a local variable named 'lpAddress'.

iv. Go to 0x1314AB0E: What registers are affected and how, when this instruction is executed?

```
0x1314AB0E                    cmp     [ebp+var_8], 102h
```

This instruction compares the value stored in a local variable with the hexadecimal value 102. If the values are found to be equal, the result of the operation is zero, and the Zero Flag (ZF) is set (1). If they are not equal, the result of the operation is non-zero, and the ZF is cleared (0).

In a 32-bit computer architecture, the ZF is part of the EFLAGS status register, and this specific instruction therefore affects said register. Other flags are altered by this instruction – such as the Carry Flag (CF) and Sign Flag (SF) – but they are also part of the EFLAGS register.

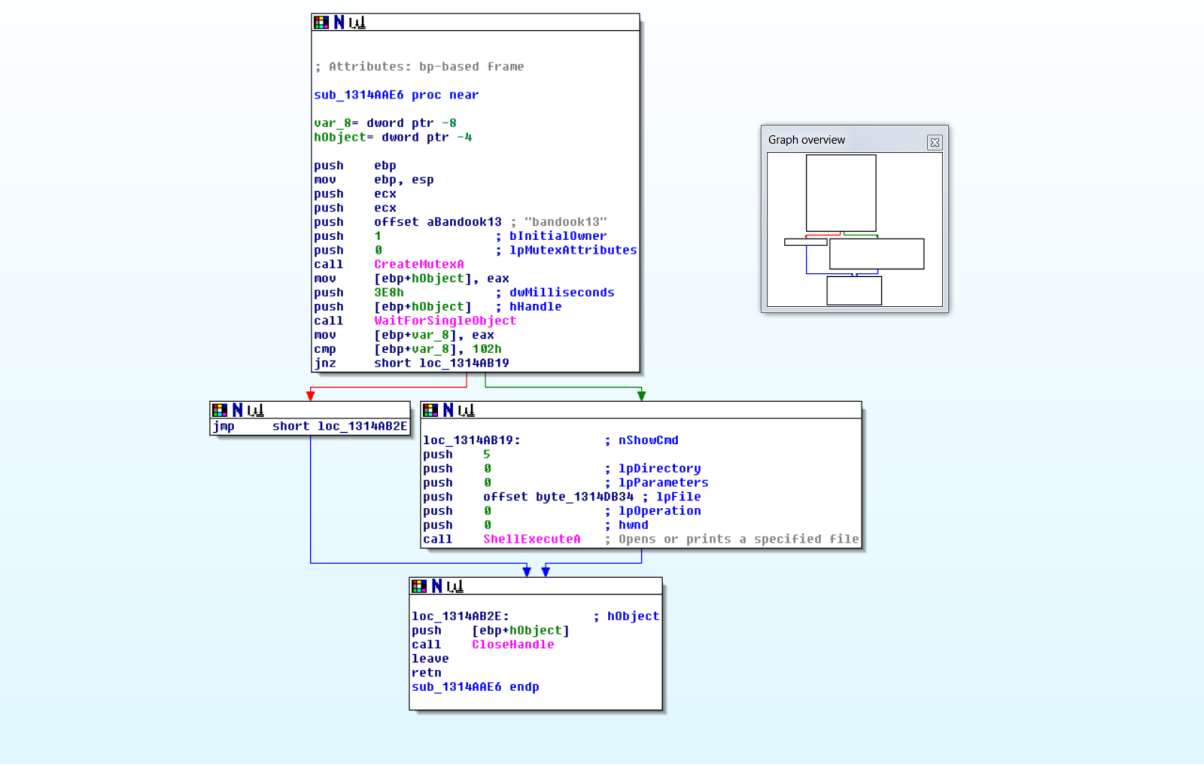
It should be noted as well that the EIP (Instruction Pointer) register is likely also affected by this instruction, as it stores/points to the address of the next instruction to be executed. This means that the EIP is naturally affected when the above instruction is executed.

v. Go to 0x1314AB15: What is tested, and what are the different outcomes?

```
0x1314AB15                    jnz     short loc_1314AB19
```

The Zero Flag (ZF) is tested, based on the comparison from the previous task. The JNZ opcode indicates that a jump will be made if the result of the comparison is not zero, which means the ZF will be cleared (value 0). This is the case if the values are found to be different. The outcome of this is that the portion of the program from location 0x1314AB19 to location 0x1314AB28 is executed, which includes a ShellExecute call that takes a collection of arguments.

If the values are found to be equal, the result of the operation is zero, which means the ZF will be set (value 1) and the jump will *not* be made. (In other words, the logic of the ZF value can easily be confused when dealing with the JNZ opcode.) The outcome of this is that the next instruction is executed instead, which is a more traditional unconditional jump that moves past the destination of the other jump and effectively skips the potentially vital call to ShellExecute. A visual representation of the decision-making process is illustrated in IDA PRO, shown below:



vi. Go to address 0x1314BD1F: What happens when this is executed?

```
0x1314BD1F                    lea     eax, [eax+ecx-80000000h]
```

Loads the effective address of [EAX+ECX-80000000h] and puts it into EAX. Note that the LEA opcode used in this particular instruction is different from the similar MOV opcode. The latter would store (into EAX) the value found at the mentioned address instead of the address itself.

vii. Go to 0x13142CEA: What is tested and what can happen?

```
0x13142CEA                    jb      short loc_13142CF3
```

The Carry Flag (CF) is tested, based on the comparison from the previous instruction. This is a comparison between the ESI register and the hexadecimal value 104. The JB opcode indicates that a jump will be made if – following the comparison – the Carry Flag (CF) is set to 1.

If the jump is made, the program moves to the location 0x13142CF3, effectively skipping only two instructions. If the jump is not made, the EAX register is cleared, before a jump is made to the location 0x13142E8D, resulting in the program skipping a great number of instructions.

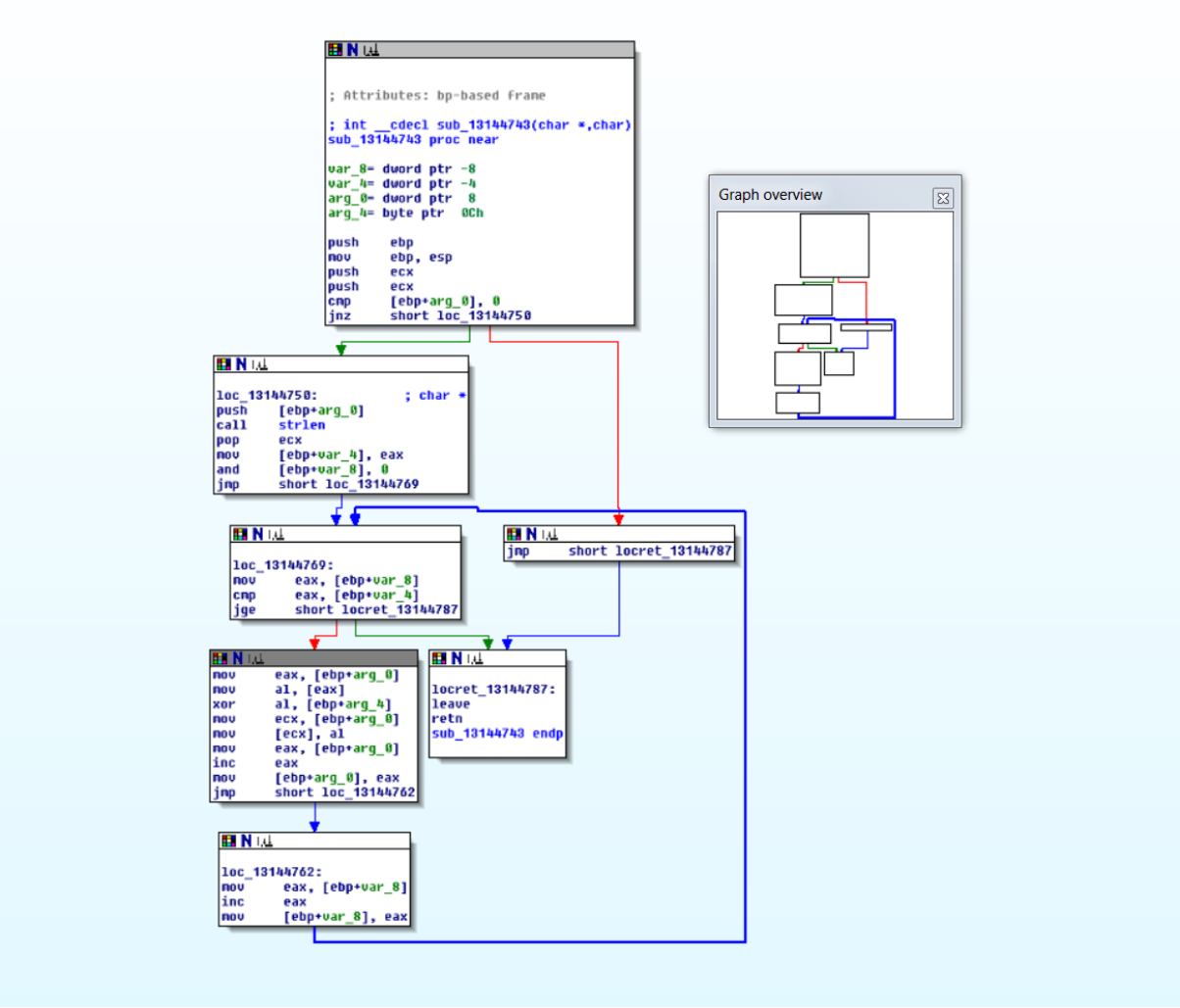
viii. Go to address 0x13142CEC: What happens when this is executed?

```
0x13142CEC                    xor     eax, eax
```

Compares the EAX register with itself, effectively clearing it by setting its value to 0. This happens because of a bit-by-bit comparison, and because of the nature of the XOR opcode. It requires that two bits are different for the resulting bit to be 1, but because each bit is always the same – after all, EAX is identical to itself – all bits will be 0, resulting in a final value of 0.

b. Malware sample: exam3

Go to the function at address 0x13144743. Provide a screenshot.



i. What input arguments are given to the function sub_13144743?

The first input argument is a pointer to a character variable, most likely representing a string. The second input argument is a normal character variable, possibly functioning as a counter.

This can be seen in the function declaration in the image above, inside the parentheses.

ii. How many local variables are used and what are they used for?

As seen in the function definition, four local variables are used in total, two of which represent the arguments given to the function (‘arg_0’ and ‘arg_4’). They are mostly used as temporary holders of data, as well as for comparisons. In addition, the argument representing a string is used as an argument to the function ‘strlen’, which precisely takes a string as an argument, calculates the length of the string, and returns the calculated length.

iii. What is the overall purpose of the function sub_13144743?

The purpose of the function seems to be to manipulate the string argument in some way, based on the length of the string and the character argument. Initially, at 0x13144748, the function compares the string argument with 0, and – if it finds that the string argument is non-existent (equals 0) – skips the JNZ and instead jumps straight to the end of the function and returns.

However, if the result of the operation is non-zero, meaning there is a string argument, the initial JNZ to 0x13144750 is taken. Here, the string argument is pushed onto the stack and used with the 'strlen' call. The returned value is the length of the string argument. Since this is stored in EAX, this is what is copied into 'var_4', before 'var_8' is nulled. What follows is a loop that each round increments 'var_8' and compares it with 'var_4' (containing the length of the string argument), looping until the counter variable reaches the length of the string argument.

c. Malware sample: exam3

Go to the address 0x131449AA.

i. What is the name of the function called at this location?

The name of the function called at this location is 'RegCreateKeyExA'. This function creates a specified registry key (unless the key already exists, in which case the function opens it).

ii. What calling convention is used?

The calling convention used is STDCALL. We can see this by examining the function definition, but a more thorough way of seeing this is by investigating who cleans up the stack when the function has completed. In STDCALL, the called function is responsible for cleaning up the stack – meaning that, in this case, 'RegCreateKeyExA' will perform the cleaning itself. This matches up with what we see here; there is no cleaning of the stack following the call to the function.

iii. What is the value of the first (leftmost) argument to the function and what does this number represent?

The value of the first (leftmost) argument to the function is what is pushed onto the stack right before the function call – ergo, 80000001 on hexadecimal form. This corresponds to a 32-bit binary string beginning with a 1, followed by thirty 0's, and ending with another 1. This number represents the registry key/hive 'HKEY_CURRENT_USER' in a 32-bit Windows architecture.

iv. What is the next value (second from left) and what is this commonly associate with when found in malware?

The next value is the registry key ‘Software\Microsoft\Windows\CurrentVersion\Run’. As we have seen in the basic dynamic analysis with tools like Regshot and Process Monitor, when found in malware, this is commonly associated with the author’s attempt to increase the persistence and capabilities of the malware, by making sure it always runs at system start-up.

d. Malware sample: exam3

Go to function call 'CreateFileA' at 0x13145059.

i. What input argument to CreateFile contains the path and the filename?

The argument pushed onto the stack immediately before the call to CreateFile contains the path and the filename. More specifically, the leftmost argument of the function (‘lpFileName’).

ii. What is the path and the filename? Explain how you found it.

The path and filename can be found by investigating what was stored into EAX before the value of this register was pushed onto the stack and used in the function call to CreateFile. Since EAX contains the return value for function calls, and the last function call was to ‘strcat’, it is reasonable to assume that this function stores the value into EAX. Going even further back, the string concatenation uses two offsets, the former containing the filename (as a result of the call to ‘sprintf’), namely ‘pwsbandook2.exe’. Thus, the other offset contains the rest of the path, given by the call to GetWindowsDirectory, which will generally return ‘C:\Windows’.

In summary:

- Filename: pwsbandook2.exe
- Path: C:\Windows\pwsbandook2.exe

e. Malware sample: exam3

Go to address 0x1314C0DB.
All questions are related to the loop from 0x1314C0DB to 0x1314C11B.

i. Explain each line of code starting from 0x1314C0DB down to and including 0x1314C11B.

Skipped due to time constraints.

ii. The value of EAX is used to decide a conditional jump 3 different places. In each of these three places; what value(s) can EAX have and what determines the value?

The three mentioned places all happen right before a conditional jump and are as follows:

```
0x1314C0DE                test    eax, eax
0x1314C0F0                test    eax, eax
0x1314C103                cmp     eax, 1
```

What determines the value of EAX in each of these three places is the instruction right before.

iii. What is the function of the loop you have analyzed?

The function of the loop appears to be to shut down a number of processes, given that a call to 'ExitProcess' is part of the loop, and sleep for half a second after each run of the loop, given that a call to 'Sleep' is part of the loop (which takes the hexadecimal value 1F4 as an argument, corresponding to the decimal value 500, measured in milliseconds, giving 0.5 seconds).

f. Malware sample: exam3

Go to the function at 0x13145928.

i. What is the purpose of the function sub_13145928? What is returned in eax from this function?

The specific value found/stored in EAX when the function returns is dependent on the code the function ends up running. What is certain though, is that the function returns an integer value (a whole number), as seen by the function declaration, which begins with "int […]".

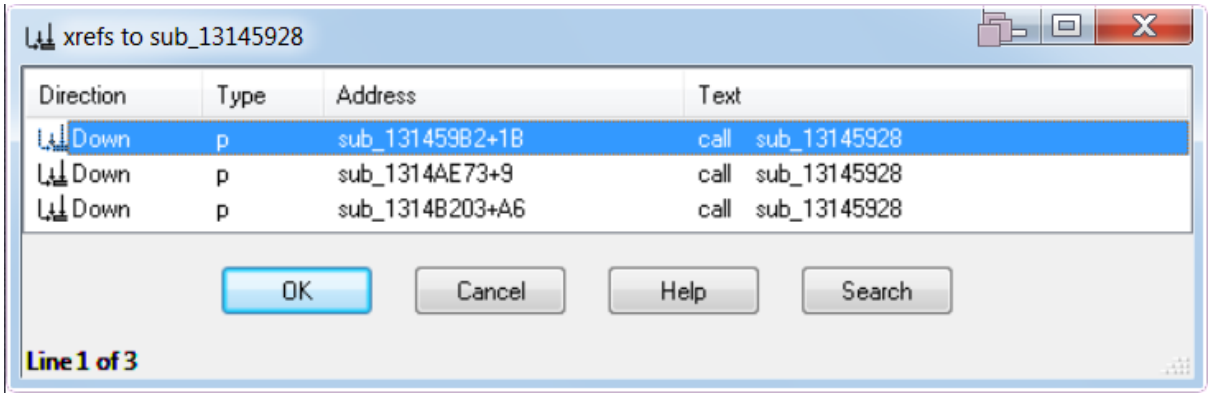
If the function gets to 0x1314596C/0x131459AE, EAX is set to 0 (due to XOR'ing itself), followed by a jump to the location of the return, and thus, 0 will be returned in EAX from this function.

If the function gets to 0x13145984, a process ID is moved into EAX, followed by a jump to the location of the return, and thus, the process ID will be returned in EAX from this function.

This last finding, revolving around process ID, could indicate the purpose of this function, which seems to be related to retrieving or manipulating the information of a specific process.

ii. What addresses are sub_13145928 called from?

To find out what addresses call this function, we can examine its cross references in IDA PRO (by marking the function name and pressing 'X'). The following window appears, which tells us the function is called from three different addresses, displayed under the "Address" column:



iii. What names are provided to the function sub_13145928 in each of the cases from the previous question? Explain how you found the answers.

Skipped due to time constraints.

g. Malware sample: exam3

What are the input argument(s) for the functions at the following addresses?

i. Function call at 0x1314B2C8

The function call is to 'Sleep', which takes one input argument. This is the hexadecimal value 3E8, equal to the decimal value 1000. Since this is measured in milliseconds (ms), the 'Sleep' function will effectively sleep for precisely one second (1000 / 1000 = 1).

ii. Function call at 0x1314B2B7

The function call is to 'OpenProcess', which takes three input arguments. From left to right, this is the hexadecimal value 1F0FFF (indicating desired access), the Boolean value 0/false (indicating inherit handle) and the value in EAX (indicating the id of the process to be opened).

iii. Function call at 0x1314A959

The function call is to 'GetProcAddress', which takes two input arguments. From left to right, this is the value in EAX – which contains the return value from the previously called function 'GetModuleHandle' – and the pushed offset at location 0x1314A948 (since the next pushed offset at location 0x1314A94D is an argument to the 'GetModuleHandle' function call).

4) Advanced Dynamic Analysis

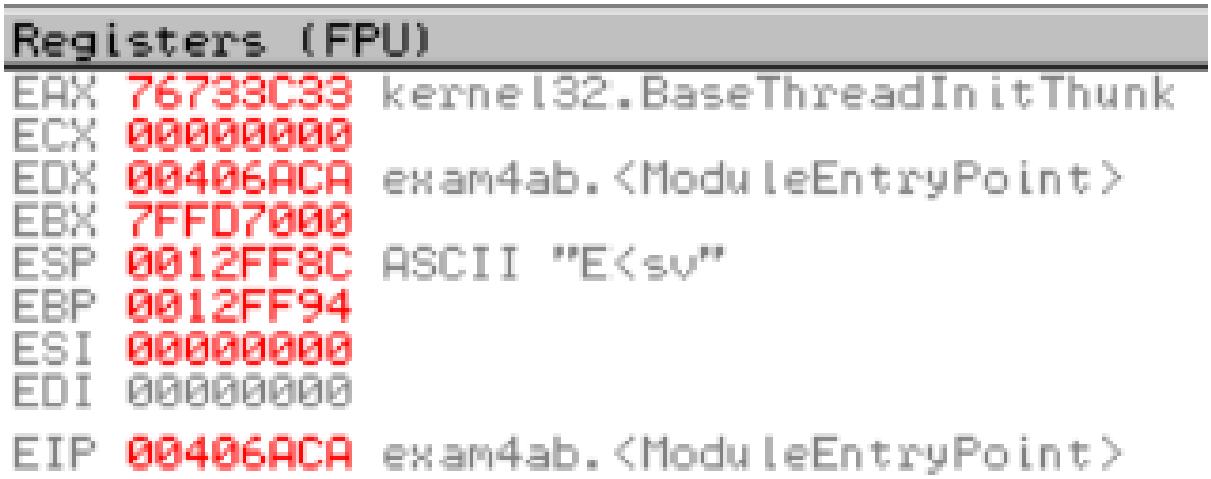
a. Malware sample: exam4ab

i. At what address and instruction does ollydbg stop?

OllyDbg stops at address 0x406ACA, with the instruction “PUSH EBP”.

ii. What is the current value of the EAX and EDX registers?

The register pane in the image below displays the current value of the EAX and EDX registers:



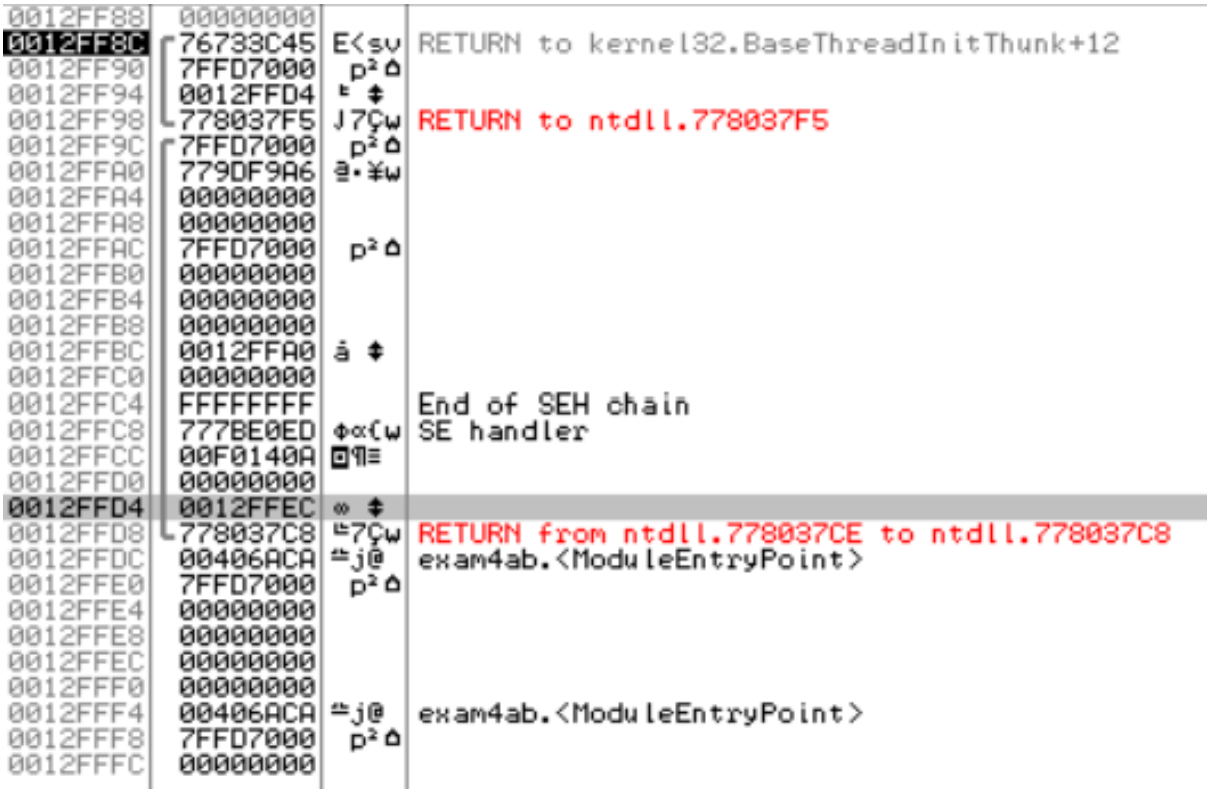
In other words, the current value of EAX and EDX is 76733C33 and 00406ACA, respectively.

iii. What is the address of the top of the stack?

The stack pointer (ESP) points to the top of the stack. As seen in the image above, the current value of the ESP register is 0012FF8C, which means this is the address of the top of the stack.

iv. What value do you find on the stack at address 12FFD4?

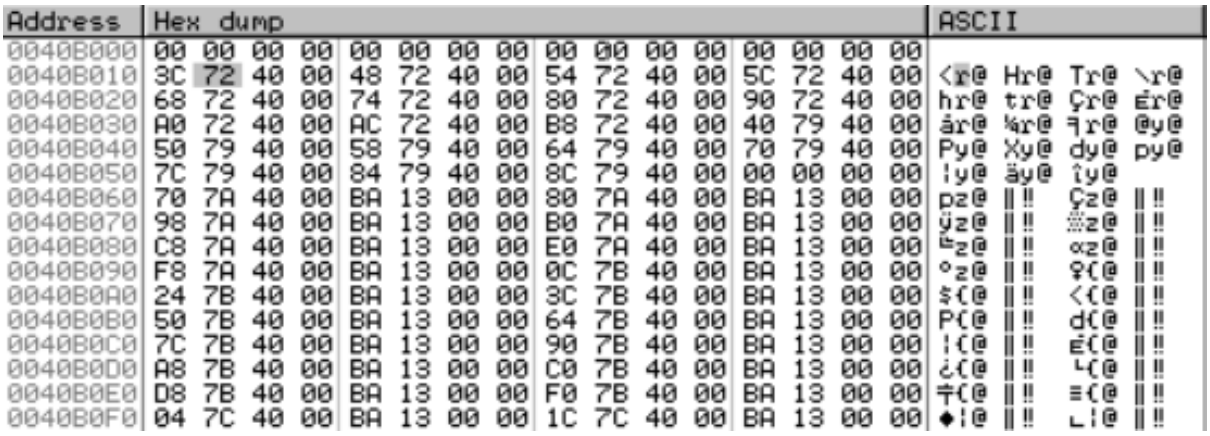
The image below displays the stack pane and value found on the stack at address 12FFD4:



Address 0012FFD4 is the line marked in grey, which shows the corresponding value 0012FFEC. However, since 0012FFD4, 0012FFD5, 0012FFD6 and 0012FFD7 all hold one byte each – where a byte corresponds to two characters in the value 0012FFEC – only two of these characters represent the value of the specific address 0012FFD4. These are the latter two characters, meaning the value found on the stack at address 0012FFD4 is the hexadecimal value EC.

v. *Use the memory view (memory dump window). What byte value do you find at address 0x40B011?*

This can be done by first navigating to the address 0x40B011 (by pressing CTRL + G and typing in the address), then right clicking the line, or one above it, and selecting "Follow in Dump" → "Selection". The image below displays the memory view from address 0x40B000 to 0x40B0FF:



The value found at address 0x40B011 is marked in grey and equals 72 on hexadecimal form (114 on decimal form). This corresponds to the byte value 0111 0010.

This precise value is found at address 0x40B011 because each two-character value in a column corresponds to one byte (eight bits) and belongs to a specific address. For example, 0x40B010 holds the first value (3C), 0x40B011 the next (72), 0x40B012 the one after that (40), and so on, up to 0x40B01F (00) – at which point, the pattern is repeated with the next line.

Execute 5 instructions.

vi. What is the current value of EBP and EIP?

After executing five instructions from where OllyDbg originally stopped – the entry point of the main module – the value of EBP and EIP have been changed to what is seen in the image below:

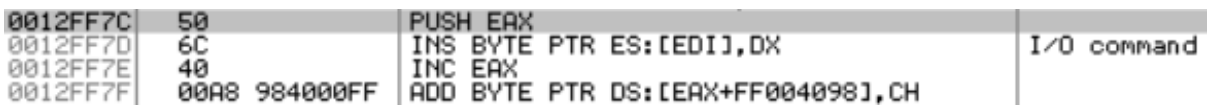


In other words, the current value of the instruction pointer (EIP) is 00406AD9, which means it now points to the instruction found at this address – that is, the one we are standing at in OllyDbg (but have not yet executed). The current value of the base pointer (EBP) is 0012FF88.

vii. What is the address of the top of the stack and what value is stored here?

The stack pointer (ESP) points to the top of the stack. As seen in the image above, the current value of the ESP register is 0012FF7C, which means this is the address of the top of the stack.

To find the value, we could do as in task iv. and use the stack pane, but another way is to simply use the disassembly pane*. Here, we go to the address 0x12FF7C (by pressing CTRL + G and typing in the address) and examine the value in the column to the right of the address. The image below displays the disassembly pane, where the value is stored next to the address:



Address 0x12FF7C is the line marked in grey, which is shown to hold the hexadecimal value 50.

* Of course, verifying that the same value is shown in the stack pane should still be done.

Go to address 0x405B39, set a break point and execute the code to this point.

viii. What instruction is located here?

Navigating to the address with the CTRL + G shortcut, followed by setting a breakpoint with F2 and executing the code to this point with F9. The instruction located at address 0x405B39 is:

```
JAE SHORT 00405B9C
```

ix. What is the value of the carrier flag?

The value of the Carry Flag (CF) is displayed in the register pane in the image below:



Here, the letter 'C' (top left) indicates the CF. The number next to it is a bit, either 0 or 1, that represents the current value of the flag. As seen in the image, the value is currently set to 1.

You are now at the beginning of a loop.

x. How many time will this loop be taken?

The number of times the loop will be taken can be found by running through the loop once, until a new comparison happens, at the instruction right before the breakpoint. At this point, the bottom window in the disassembly pane shows us that the loop will be taken 2382 times from a hexadecimal perspective, which corresponds to 9090 times in total.

xi. At what address is the value deciding the number of loops stored?

The address deciding the number of loops stored is found in the same window described in the previous task and is shown to be the location 0x12FAC0. Since this specific address technically only holds one byte (0x12FAC1 holds the next byte, and so on), and 2382 hex corresponds to two bytes, the number is also partly stored at the address directly after – 0x12FAC1.

xii. Take a screenshot showing this value in memory view (hex dump) and in the stack view simultaneously.

Note: I am assuming here that they have to be shown together, and that two separately taken images side-by-side is not sufficient. Therefore, the image below is difficult to interpret/view.

A simultaneous screenshot of the value in memory view and stack view is displayed below:



The value is shown at the topmost line in both views (← memory view / stack view →).

xiii. What do we call the representation used for the value in memory and what do we call the representation used on the stack?

Memory: First-In-First-Out (FIFO). The bytes are "sorted" differently to how we would normally read them, as seen by the fact that the hexadecimal value 2383 is represented as 83 23.

Stack: Last-In-First-Out (LIFO). The bytes are "sorted" similarly to how we would normally read them, as seen by the fact that the hexadecimal value 2383 is represented as 23 83.

Alternatively, a common saying is that the stack "grows towards lower addresses".

b. Malware sample: exam4ab

i. Provide the name and the address of the instruction that executes the .bat file.

Step-by-step: Switch to the memory map window (ALT + M) and search for the ASCII string ".bat" (CTRL + B) in address sections where the owner is 'exam4ab'. This generates two results, one at 0x407870 and one at 0x407A00 – both in the '.rdata' section. From here, we need to figure out where in memory the mentioned addresses are used, which can be done by right clicking the selected ".bat" string in each dump window and choosing "Find references to" → "Selection". This displays one reference for each, one at 0x403026 and the other at 0x404554.

Going back to the main window and looking up these two addresses in the disassembly pane shows that one of them is particularly interesting, namely the latter. Here, the extension ".bat" is used as one of two arguments to a function, followed by calls to 'fopen' (with the argument 'w', indicating writing) and 'ShellExecute' (with an argument 'open', indicating file execution) further down. Note that none of this happens at the other location, leaving it less relevant.

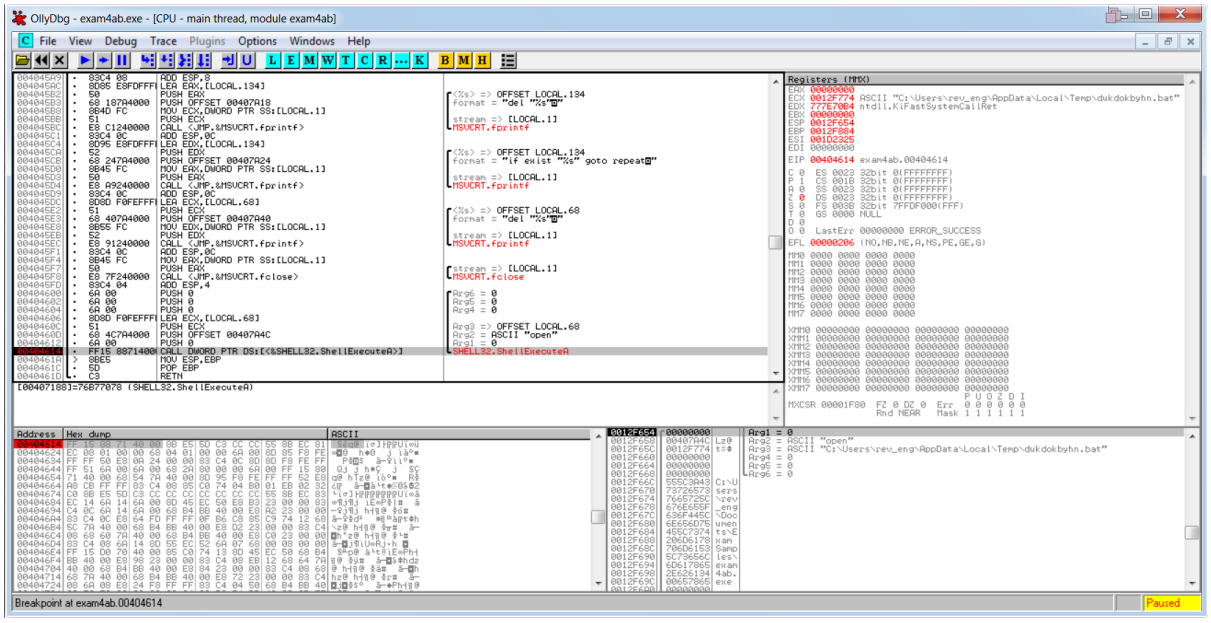
A screenshot of the mentioned location, from 0x404554 to 0x404614, is provided below:

```
00404554  .  68 007A4000    PUSH OFFSET 00407A00                     ; Arg2 = ASCII ".bat"
00404559  .  8D95 F0FEFFFF  LEA EDX,[LOCAL.68]
0040455F  .  52             PUSH EDX                                 ; Arg1 => OFFSET LOCAL.68
00404560  .  E8 2F250000    CALL <JMP.&MSVCRT.strcat>                ; MSVCRT._mbscat
00404565  .  83C4 08        ADD ESP,8
00404568  .  68 087A4000    PUSH OFFSET 00407A08                     ; mode = "w"
0040456D  .  8D85 F0FEFFFF  LEA EAX,[LOCAL.68]
00404573  .  50             PUSH EAX                                 ; filename => OFFSET LOCAL.68
00404574  .  E8 15250000    CALL <JMP.&MSVCRT.fopen>                 ; MSVCRT.fopen
00404579  .  83C4 08        ADD ESP,8
0040457C  .  8945 FC        MOV DWORD PTR SS:[LOCAL.1],EAX
0040457F  .  837D FC 00     CMP DWORD PTR SS:[LOCAL.1],0
00404583  .v 0F84 91000000  JE 0040461A
00404589  .  68 80000000    PUSH 80                                  ; Attributes = FILE_ATTRIBUTE_NORMAL
0040458E  .  8D8D E8FDFFFF  LEA ECX,[LOCAL.134]
00404594  .  51             PUSH ECX                                 ; Name => OFFSET LOCAL.134
00404595  .  FF15 3C704000  CALL DWORD PTR DS:[<&KERNEL32.SetFileAttributesA>] ; KERNEL32.SetFileAttributesA
0040459B  .  68 0C7A4000    PUSH OFFSET 00407A0C                     ; format = ":repeat"
004045A0  .  8B55 FC        MOV EDX,DWORD PTR SS:[LOCAL.1]
004045A3  .  52             PUSH EDX                                 ; stream => [LOCAL.1]
004045A4  .  E8 D9240000    CALL <JMP.&MSVCRT.fprintf>               ; MSVCRT.fprintf
004045A9  .  83C4 08        ADD ESP,8
004045AC  .  8D85 E8FDFFFF  LEA EAX,[LOCAL.134]
004045B2  .  50             PUSH EAX                                 ; <%s> => OFFSET LOCAL.134
004045B3  .  68 187A4000    PUSH OFFSET 00407A18                     ; format = "del "%s""
004045B8  .  8B4D FC        MOV ECX,DWORD PTR SS:[LOCAL.1]
004045BB  .  51             PUSH ECX                                 ; stream => [LOCAL.1]
004045BC  .  E8 C1240000    CALL <JMP.&MSVCRT.fprintf>               ; MSVCRT.fprintf
004045C1  .  83C4 0C        ADD ESP,0C
004045C4  .  8D95 E8FDFFFF  LEA EDX,[LOCAL.134]
004045CA  .  52             PUSH EDX                                 ; <%s> => OFFSET LOCAL.134
004045CB  .  68 247A4000    PUSH OFFSET 00407A24                     ; format = "if exist "%s" goto repeat"
004045D0  .  8B45 FC        MOV EAX,DWORD PTR SS:[LOCAL.1]
004045D3  .  50             PUSH EAX                                 ; stream => [LOCAL.1]
004045D4  .  E8 A9240000    CALL <JMP.&MSVCRT.fprintf>               ; MSVCRT.fprintf
004045D9  .  83C4 0C        ADD ESP,0C
004045DC  .  8D8D F0FEFFFF  LEA ECX,[LOCAL.68]
004045E2  .  51             PUSH ECX                                 ; <%s> => OFFSET LOCAL.68
004045E3  .  68 407A4000    PUSH OFFSET 00407A40                     ; format = "del "%s""
004045E8  .  8B55 FC        MOV EDX,DWORD PTR SS:[LOCAL.1]
004045EB  .  52             PUSH EDX                                 ; stream => [LOCAL.1]
004045EC  .  E8 91240000    CALL <JMP.&MSVCRT.fprintf>               ; MSVCRT.fprintf
004045F1  .  83C4 0C        ADD ESP,0C
004045F4  .  8B45 FC        MOV EAX,DWORD PTR SS:[LOCAL.1]
004045F7  .  50             PUSH EAX                                 ; stream => [LOCAL.1]
004045F8  .  E8 7F240000    CALL <JMP.&MSVCRT.fclose>                ; MSVCRT.fclose
004045FD  .  83C4 04        ADD ESP,4
00404600  .  6A 00          PUSH 0                                   ; Arg6 = 0
00404602  .  6A 00          PUSH 0                                   ; Arg5 = 0
00404604  .  6A 00          PUSH 0                                   ; Arg4 = 0
00404606  .  8D8D F0FEFFFF  LEA ECX,[LOCAL.68]
0040460C  .  51             PUSH ECX                                 ; Arg3 => OFFSET LOCAL.68
0040460D  .  68 4C7A4000    PUSH OFFSET 00407A4C                     ; Arg2 = ASCII "open"
00404612  .  6A 00          PUSH 0                                   ; Arg1 = 0
00404614  .  FF15 88714000  CALL DWORD PTR DS:[<&SHELL32.ShellExecuteA>] ; SHELL32.ShellExecuteA
```

To conclude, the address of the instruction that executes the ".bat" file is 0x404614, while the name of the instruction is "CALL DWORD […]", shown at the very bottom of the image above.

ii. Provide a screenshot from ollydbg at this point. The .bat filename should be visible in memory and stack view, as well as a register.

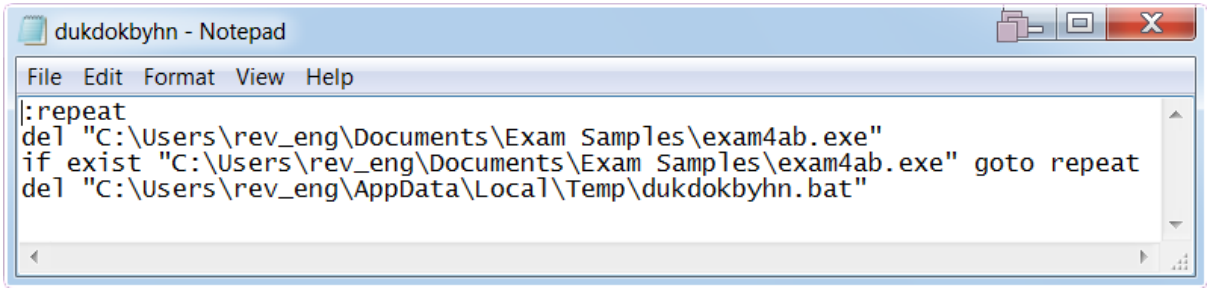
Screenshot after having run the code to the instruction that executes the ".bat" file is below:



As seen from the stack (line three from the top) and register (ECX) views, the newly created file has the name “dukdokbyhn.bat”, with location “C:\Users\rev_eng\AppData\Local\Temp\”.

iii. Open the .bat file in a text editor and provide a screenshot of its content. Explain what the bat file is doing.

The content of the “.bat” file is shown below, matching up with the ‘fprintf’ calls seen earlier:

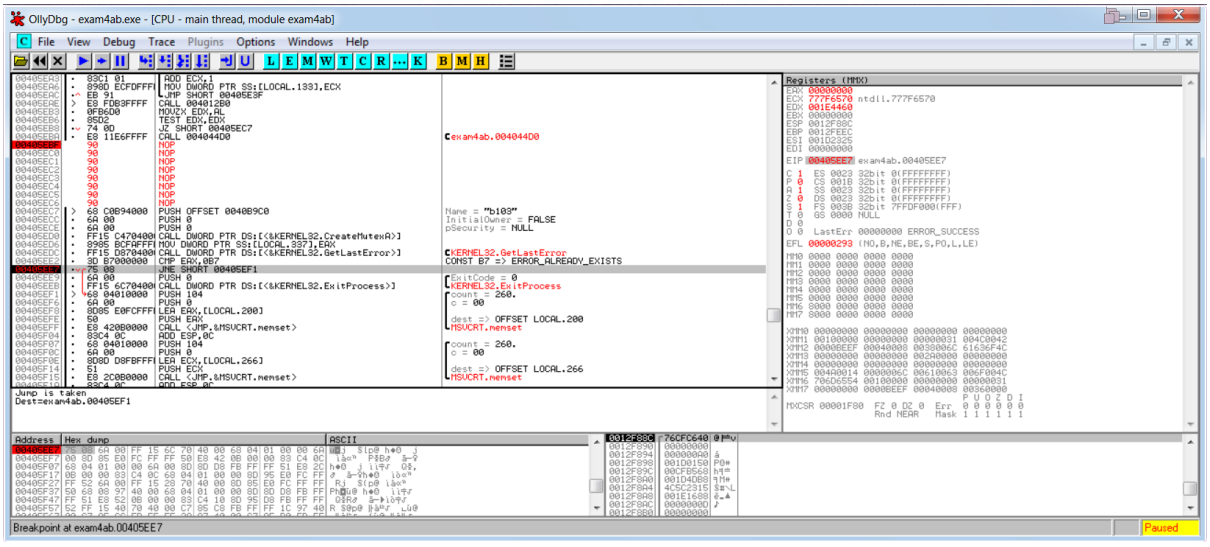


It appears that this is a script that, once executed, runs in a loop, each time attempting to delete the original sample “exam4ab.exe”, and – if the deletion was unsuccessful, meaning the sample still exists afterward – goes back and repeats the attempt. If, on the other hand, it finds that the deletion of the original sample was successful, it moves on to the final line of the script, where it also deletes itself, perhaps indicating that it tries to erase traces of the malware.

Go to address 0x405EBF, set a breakpoint and run to this address. Replace the instruction at 0x405EBF and 0x405EC1 with NOPs.

iv. Execute instructions until you are at 0x405EE7 and then provide a screenshot.

We can replace the instructions with no-operations (NOPs) by right-clicking each instruction and selecting “Edit” → “Fill with NOPs”. Below is a screenshot after everything has been done:



v. Explain what decides if the jump is taken or not. What is the likely purpose of this?

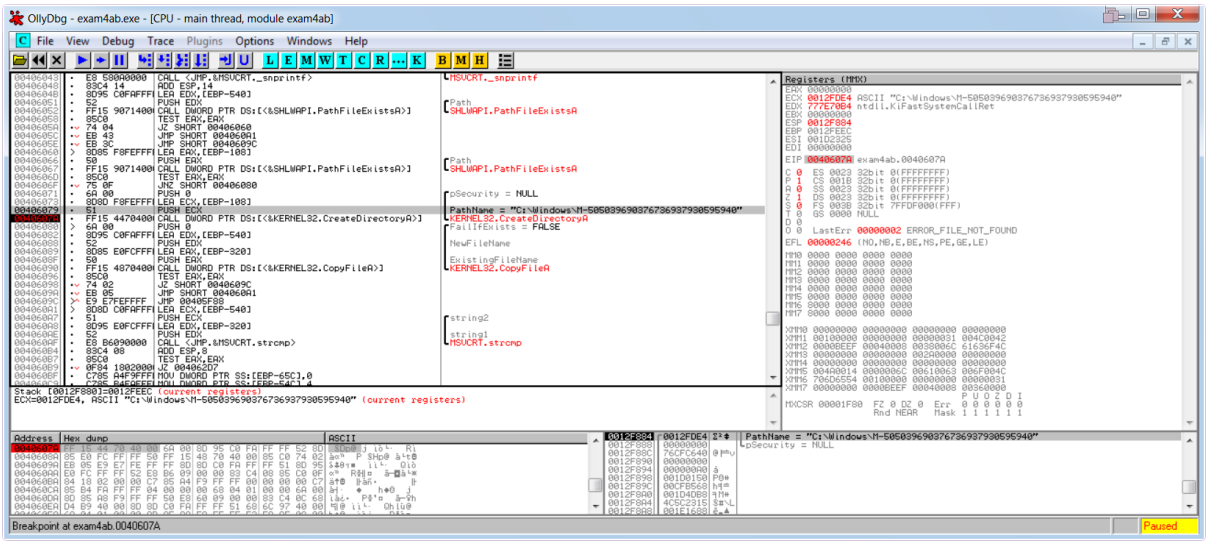
The comparison made in the instruction above the jump, at 0x405EE2, decides if the jump is taken or not. If the values in the comparison are found to be different, that is, not equal, the conditional jump is performed. On the other hand, if the values in the comparison are found to be equal, a call to 'ExitProcess' is made. The purpose of this is likely to prevent the malware from running more than once. This theory is supported by a couple of clear indicators:

1) The comparison is between the content of EAX (where the return value from 'GetLastError' has been stored) and B7 hex, which represents the error "ERROR_ALREADY_EXISTS". Ergo, if the value in EAX corresponds to this, the jump is not taken, and the process is exited.
2) Right before the call to 'GetLastError' is a call to 'CreateMutex', meaning there is a mutex involved, the purpose of which normally is to ensure a program only runs once at a time.

Go to address 0x40607A, set breakpoint and run to this point.

vi. What directory is created here? Provide screenshot.

The directory created here is "C:\Windows\M-50503969037673693793059594O", as seen in the disassembly (line marked grey), register (ECX) and stack (top line) panes in this screenshot:



vii. What file is copied in 0x406090, provide full path for both source and destination file.

This can be found out by setting a breakpoint at 0x406090, right before the call to CopyFile, then running to this point, and finally examining the values that appear in the various panes.

Source file path: C:\Users\rev_eng\Documents\Exam Samples\exam4ab.exe
Destination file path: C:\Windows\M-50503969037673693793059594O\winmgr.exe

In other words, the file copied is the sample currently being analysed, 'exam4ab.exe', meaning it copies itself. This is copied to the obscurely named sub-directory located inside the Windows directory, possibly to increase its own persistence and/or capabilities. On a side note, both the

created directory from the previous task (with the obscure name) and the file 'winmgr.exe' are familiar names, as these were also created by the first sample in the basic dynamic analysis.

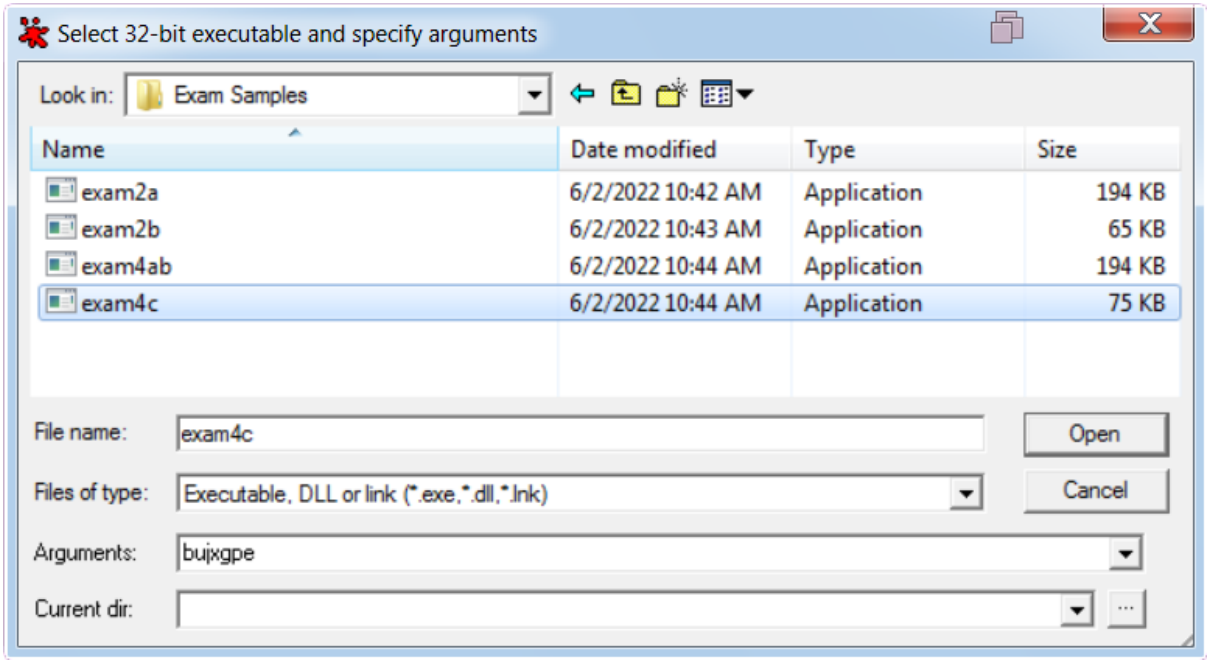
viii. What happens to the folder and the file when the instructions SetFileAttributes are executed in 0x406117 and 0x406126?

When these instructions are executed, the attributes of the folder* and file* are set to be read-only and hidden. In combination with the deletion of the original sample (discussed in task iii.), this is probably the malware's attempt to remove traces of itself and hide its own existence in the infected system. In addition, this likely explains why, during the basic dynamic analysis (assuming sample "exam2a" also does this), the obscurely named directory was never visible in file explorer, even after showing hidden files and folders (as discussed in task 2.a.ii.).

* This of course refers to the folder and file that were created in the previous two tasks.

c. Malware sample: exam4c and bujxgpe

Make sure that the bujxgpe-file is in the same directory as exam4c. Open exam4c in ollydbg with bujxgpe as an argument. Place a breakpoint at address 0x401000 and run until you hit it.



i. Run until the line after "call VirtualAlloc". What is the address to the memory-space allocated by VirtualAlloc? You will need this baseaddress in the following questions.

When the 'VirtualAlloc' function succeeds, it returns the base-address of the memory-space it allocated. Therefore, after running until the line after the call to this function, the address can

be found by examining the EAX register, since this is where return values from function calls are stored. Hence, the address to the memory-space allocated by the function is 0x2D0000.

ii. Run until the call to EnumSystemCodePageW. Go to the address passed as the first argument to this API call. What is this address and what is the instruction here?

The address passed as the first argument to this API call is the same address returned by the function in the previous task, namely 0x2D0000. The instruction located at this address is:

```
JMP 002D0809
```

iii. Set a breakpoint where you are (if you get a warning, press yes) and run to it. Step down to/run to [baseaddress + 879h]. Baseaddress is your address from the first question (i). You will see a number of calls to a function [baseaddress + 7A4h]. Change the addressing of ollydbg's stackview to "relative to EBP". Step through at least 8 of the function calls. Provide a screenshot of the stackview, showing the results of the calls. What is the purpose of this function [baseaddress + 7A4h]?

After stepping through all of the calls – precisely ten – to the function [base-address + 7A4h], in this case 002D07A4, the stack view in the screenshot below shows the results of the calls:

```
EBP-484 | 759FCF66 | f=ƒu | RETURN from KERNELBASE.759FCE78 to KERNELBASE.759FCF66
EBP-480 | 00000000 |
EBP-47C | 7FFFFFF2 | ≥  ⌂
EBP-478 | 0012FC94 | ö"$
EBP-474 | 00000200 |  ▼
EBP-470 | 777E5D14 | ¶]~w | RETURN to ntdll.NtOpenKey+0C
EBP-46C | 759FD127 | '=ƒu | RETURN from ntdll.NtOpenKey to KERNELBASE.OpenRegKey+0D8
EBP-468 | 0012FC94 | ö"$
EBP-464 | 00020019 | ↓ ▼
EBP-460 | 0012F84C | L°$
EBP-45C | 00000000 |
EBP-458 | 00000000 |
EBP-454 | 759FD161 | a=ƒu | RETURN from KERNELBASE.759F67F4 to KERNELBASE.OpenRegKey+112
EBP-450 | 0080007E | ~ Ç
EBP-44C | 0012F874 | t°$ | UNICODE "\Registry\Machine\System\CurrentControlSet\Control\Nls\Codepage"
EBP-448 | 00000018 | ↑
EBP-444 | 00000000 |
EBP-440 | 0012F844 | D°$
EBP-43C | 00000040 | @
EBP-438 | 00000000 |
EBP-434 | 00000000 |
EBP-430 | 002F3D48 | H=/
EBP-42C | 00000000 |
EBP-428 | 75A178C0 | └x¡u | UNICODE "\Nls\Codepage"
EBP-424 | 00000000 |
```

iv. Use the base memory address from question 1; go to the address of [base-address + 9B5h]. What is the first argument passed to this function call?

The address [base-address + 9B5h] corresponds to 002D09B5 in this case. To see arguments, we navigate to this address, set a breakpoint, and run to this point. The first argument passed to the function call is pushed onto the stack right before the call, at 002D09B4. The instruction here is "PUSH EAX", which means that the argument will show up in the EAX register. Inspecting this register informs us that the first argument passed to the function call is the file name/path:

C:\Users\rev_eng\AppData\Local\Temp\br6ewlh02z82va

v. *Go to address [base-address + CFh] (you won't be able to run here). Analyze this and the next line of assembly; What is it trying to do, and why?*

The address [base-address + CFh] corresponds to 002D00CF in this case. The instruction found here appears to be a call to the next line of assembly, at 002D00D4. The instruction found here seems to add the 32-bit integer representation of 5 to the address of the stack pointer (ESP).

This would effectively remove the top five bytes from the stack and is usually done to clean-up the stack after a function has finished. In general, the convention 'ADD ESP […]' – which is close to what we see here – is used for restoring the stack and moving past arguments pushed onto the stack at a function call. Ergo, this line of assembly might be part of a function epilogue.